Chương 7

Tiền huấn luyện

Sự phát triển của các mô hình chuỗi dựa trên mạng nơ-ron, chẳng hạn Transformer, cùng với những tiến bộ trong học tự giám sát (*self-supervised learning*) quy mô lớn, đã mở ra cánh cửa cho việc hiểu và sinh ngôn ngữ một cách phổ quát. Thành tựu này phần lớn có được nhờ tiền huấn luyện (*pre-training*): ta tách ra những thành phần chung từ nhiều hệ thống dựa trên mạng nơ-ron rồi huấn luyện chúng trên một lượng khổng lồ dữ liệu chưa gán nhãn bằng cơ chế tự giám sát. Các mô hình đã được tiền huấn luyện này đóng vai trò là mô hình nền tảng (*foundation model*), có thể dễ dàng được thích nghi cho nhiều nhiệm vụ khác nhau thông qua tinh chỉnh (*fine-tuning*) hoặc kỹ thuật prompt (*prompting*). Nhờ đó, mô thức của NLP đã thay đổi một cách căn bản. Trong nhiều trường hợp, ta không còn cần đến học có giám sát (*supervised learning*) quy mô lớn cho từng nhiệm vụ cụ thể nữa, mà chỉ cần thích nghi các mô hình nền tảng đã được tiền huấn luyện.

Mặc dù tiền huấn luyện mới trở nên phổ biến trong các nghiên cứu NLP gần đây, khái niệm này đã có từ hàng chục năm trước, từ những ngày đầu của học sâu (*deep learning*). Ví dụ, những nỗ lực ban đầu nhằm tiền huấn luyện các hệ thống học sâu bao gồm học không giám sát cho RNN, cho các mạng truyền thẳng sâu, cho bộ tự mã hóa (*autoencoder*), v.v. [Schmidhuber, 2015]. Trong kỷ nguyên học sâu hiện đại, tiền huấn luyện đã hồi sinh, một phần nhờ việc huấn luyện không giám sát quy mô lớn các mô hình nhúng từ (*word embedding*) khác nhau [Mikolov và cộng sự, 2013c; Pennington và cộng sự, 2014]. Cũng trong giai đoạn đó, tiền huấn luyện thu hút sự quan tâm lớn trong lĩnh vực thị giác máy tính, nơi

các mô hình xương sống (*backbone model*) được huấn luyện trên những bộ dữ liệu có nhãn tương đối lớn như ImageNet, rồi được áp dụng cho các nhiệm vụ hạ nguồn (*downstream task*) khác nhau [He và cộng sự, 2019; Zoph và cộng sự, 2020]. Nghiên cứu quy mô lớn về tiền huấn luyện trong NLP bắt đầu cùng với sự phát triển của các mô hình ngôn ngữ (*language model*) sử dụng học tự giám sát. Họ mô hình này gồm nhiều ví dụ nổi tiếng như BERT [Devlin và cộng sự, 2019] và GPT [Brown và cộng sự, 2020], tất cả đều có chung một ý tưởng: khả năng hiểu và sinh ngôn ngữ tổng quát có thể đạt được bằng cách huấn luyện mô hình dự đoán các từ bị che hoặc các từ tiếp theo trong một lượng văn bản khổng lồ. Bất chấp sự đơn giản của cách tiếp cận này, các mô hình thu được cho thấy khả năng đáng kinh ngạc trong việc nắm bắt cấu trúc ngôn ngữ, ngay cả khi không được huấn luyện tường minh để làm điều đó. Tính tổng quát của các nhiệm vụ tiền huấn luyện dẫn đến những hệ thống có hiệu năng mạnh trên rất nhiều bài toán NLP, thậm chí vượt qua các hệ thống có giám sát tiên tiến nhất (*state-of-the-art*) trước đó. Gần đây hơn, các mô hình ngôn ngữ lớn (*large language model*) được tiền huấn luyện đã đạt được thành công còn lớn hơn, cho thấy triển vọng đầy hứa hẹn hướng tới trí tuệ nhân tạo tổng quát hơn [Bubeck và cộng sự, 2023].

Chú giải của người dịch: Tiền huấn luyện – hình dung đơn giản

Hãy nghĩ tới việc học của một học sinh. Trước khi học chuyên sâu một nghề (ví dụ thi vào ngành y), em đã trải qua 12 năm học phổ thông: đọc rất nhiều, học ngôn ngữ, toán, kiến thức chung về thế giới. Giai đoạn "học nền" này giống với *tiền huấn luyện*: mô hình đọc hàng tỉ câu văn để học cách ngôn ngữ vận hành. Sau đó, việc học một nhiệm vụ cụ thể (phân loại cảm xúc, dịch máy…) chỉ cần một ít dữ liệu và công sức thêm — giống như học chuyên ngành trên nền kiến thức phổ thông đã có. Đó là *tinh chỉnh* (hoặc dùng *prompt*). Một mô hình được tiền huấn luyện tốt, dùng làm "nền móng" cho nhiều ứng dụng, được gọi là *mô hình nền tảng*.

"Dữ liệu chưa gán nhãn" là văn bản thô (bài báo, sách, trang web) không có ai ghi chú đáp án; còn "dữ liệu có nhãn" là dữ liệu kèm đáp án do con người ghi, ví dụ mỗi câu bình luận kèm nhãn "tích cực" hay "tiêu cực". Dữ liệu có nhãn tốn công và đắt, còn văn bản thô thì có sẵn rất nhiều trên Internet.

Chương này bàn về khái niệm tiền huấn luyện trong bối cảnh NLP. Chương bắt đầu bằng phần giới thiệu chung về các phương pháp tiền huấn

luyện và ứng dụng của chúng. Sau đó, BERT được dùng làm ví dụ để minh họa cách một mô hình chuỗi được huấn luyện thông qua một nhiệm vụ tự giám sát, gọi là **mô hình hóa ngôn ngữ có che** (*masked language modeling*). Tiếp theo là phần thảo luận về các phương pháp thích nghi các mô hình chuỗi đã tiền huấn luyện cho nhiều nhiệm vụ NLP khác nhau. Vì chương này tập trung chủ yếu vào mô thức tiền huấn luyện, chúng tôi sẽ để phần thảo luận chi tiết về các mô hình ngôn ngữ lớn dạng sinh sang các chương sau.

7.1 Tiền huấn luyện trong NLP

Tiền huấn luyện trong NLP thường liên quan đến hai loại nhiệm vụ: mô hình hóa chuỗi (*sequence modeling*) (hay mã hóa chuỗi (*sequence encoding*)) và sinh chuỗi (*sequence generation*). Mặc dù hai loại nhiệm vụ này có hình thức khác nhau, để đơn giản, ta mô tả chúng bằng một mô hình hợp nhất duy nhất được định nghĩa như sau:

$$\begin{aligned}\mathbf{o} &= g(x_0, x_1, \cdots, x_m; \theta) \\ &= g_\theta(x_0, x_1, \cdots, x_m),\end{aligned} \tag{7.1}$$

trong đó $\{x_0, x_1, \ldots, x_m\}$ ký hiệu một chuỗi token đầu vào[1], x_0 là một ký hiệu đặc biệt ($\langle s \rangle$ hoặc [CLS]) được gắn vào đầu chuỗi, $g(\cdot; \theta)$ (cũng được viết là $g_\theta(\cdot)$) ký hiệu một mạng nơ-ron (*neural network*) với tham số (*parameter*) θ, và $\mathbf{o}$ ký hiệu đầu ra của mạng nơ-ron. Dạng của đầu ra $\mathbf{o}$ thay đổi tùy theo bài toán cụ thể. Ví dụ, trong các nhiệm vụ dự đoán token (như mô hình hóa ngôn ngữ), $\mathbf{o}$ là một phân phối xác suất (*probability distribution*) trên một bộ từ vựng (*vocabulary*); trong các nhiệm vụ mã hóa chuỗi, $\mathbf{o}$ là một biểu diễn (*representation*) của chuỗi đầu vào, thường được biểu diễn dưới dạng một chuỗi các vectơ số thực.

Chú giải của người dịch: Đọc Phương trình (7.1)

Hãy xem g như một "cỗ máy" (một hàm số rất phức tạp): đầu vào là dãy token $x_0, x_1, \ldots, x_m$ (ví dụ các từ của một câu), đầu ra là $\mathbf{o}$. Hình dạng cụ thể của

[1] Ở đây ta giả định rằng token là đơn vị cơ bản của văn bản được tạo ra bởi quá trình tách token (*tokenization*). Đôi khi, chúng tôi dùng các thuật ngữ *token* và *từ* (*word*) thay thế cho nhau, mặc dù trong NLP chúng có nghĩa gần nhau nhưng hơi khác nhau.

cỗ máy được quyết định bởi các "núm vặn" θ — đó là tham số, thường gồm hàng triệu đến hàng tỉ con số. Giống như hàm $y = ax + b$ ở THPT: a, b là tham số, đổi a, b thì đổi hàm. Cách viết $g(\ldots;\theta)$ hay $g_\theta(\ldots)$ chỉ để nhấn mạnh rằng hàm g phụ thuộc vào θ. "Huấn luyện" chính là đi tìm bộ θ tốt. Token x_0 là một "ký hiệu mở đầu" quy ước (ví dụ [CLS]), không phải một từ thật.

Có hai khía cạnh cơ bản cần xem xét ở đây:

- *Tối ưu hóa θ cho một nhiệm vụ tiền huấn luyện.* Khác với các bài toán học tiêu chuẩn trong NLP, tiền huấn luyện không giả định trước các nhiệm vụ hạ nguồn cụ thể mà mô hình sẽ được áp dụng. Thay vào đó, mục tiêu là huấn luyện một mô hình có thể tổng quát hóa (*generalize*) trên nhiều nhiệm vụ khác nhau.
- *Áp dụng mô hình đã tiền huấn luyện $g_{\hat{\theta}}(\cdot)$ cho các nhiệm vụ hạ nguồn.* Để thích nghi mô hình với các nhiệm vụ cụ thể này, ta cần tinh chỉnh các tham số $\hat{\theta}$ bằng dữ liệu có nhãn, hoặc prompt mô hình bằng các mô tả nhiệm vụ.

Trong mục này, chúng ta bàn về các khái niệm cơ bản để giải quyết hai khía cạnh trên.

7.1.1 Tiền huấn luyện không giám sát, có giám sát và tự giám sát

Trong học sâu, tiền huấn luyện là quá trình tối ưu hóa một mạng nơ-ron trước khi nó được tinh chỉnh thêm và áp dụng cho các nhiệm vụ hạ nguồn. Cách tiếp cận này dựa trên giả định rằng một mô hình được tiền huấn luyện cho một nhiệm vụ có thể được thích nghi hiệu quả để thực hiện một nhiệm vụ khác. Nhờ vậy, ta không cần huấn luyện một mạng nơ-ron sâu và phức tạp từ đầu cho các nhiệm vụ có ít dữ liệu gán nhãn. Thay vào đó, ta có thể tận dụng những nhiệm vụ mà tín hiệu giám sát dễ thu thập hơn. Điều này làm giảm sự phụ thuộc vào việc gán nhãn thủ công của con người cho từng nhiệm vụ, cho phép phát triển những mô hình tổng quát hơn, không bị bó hẹp trong các bài toán cụ thể.

Trong thời kỳ hồi sinh của mạng nơ-ron nhờ học sâu, nhiều nỗ lực tiền huấn luyện ban đầu tập trung vào **học không giám sát** (*unsupervised*

learning). Trong các phương pháp này, tham số của mạng nơ-ron được tối ưu hóa theo một tiêu chí không liên quan trực tiếp đến các nhiệm vụ đích cụ thể. Ví dụ, một mô hình có thể được huấn luyện để cực tiểu hóa entropy chéo (*cross-entropy*) tái tạo của vectơ đầu vào ở mỗi tầng (*layer*) [Bengio và cộng sự, 2006]. Tiền huấn luyện không giám sát thường được dùng như một bước sơ bộ trước học có giám sát, mang lại nhiều lợi ích, chẳng hạn giúp tìm ra các cực tiểu địa phương (*local minima*) tốt hơn và đóng vai trò như một bộ chính quy hóa (*regularizer*) trong quá trình huấn luyện [Erhan và cộng sự, 2010]. Những lợi ích này khiến giai đoạn học có giám sát sau đó dễ dàng và ổn định hơn.

Cách tiếp cận thứ hai là tiền huấn luyện mạng nơ-ron trên các nhiệm vụ **học có giám sát**. Ví dụ, xét một mô hình chuỗi được thiết kế để mã hóa các chuỗi đầu vào thành các biểu diễn vectơ. Trong quá trình tiền huấn luyện, mô hình này được kết hợp với một tầng phân loại và được huấn luyện trên một nhiệm vụ có giám sát ban đầu, chẳng hạn phân tích cảm xúc (*sentiment analysis*) (ví dụ, phân loại một câu là tích cực hay tiêu cực). Để thích nghi mô hình chuỗi đã tiền huấn luyện cho một nhiệm vụ hạ nguồn (ví dụ, xác định một chuỗi mang tính chủ quan hay khách quan), ta thay tầng phân loại ban đầu bằng một tầng mới cho nhiệm vụ đích. Thông thường, ta cần tinh chỉnh các tham số của mô hình kết hợp này bằng dữ liệu có nhãn riêng của nhiệm vụ để tối ưu hiệu năng. Mô hình đã tinh chỉnh sau đó được dùng để phân loại các chuỗi mới. Một ưu điểm lớn của tiền huấn luyện có giám sát là nó khớp một cách trực tiếp với các mô thức học có giám sát tiêu chuẩn. Tuy nhiên, khi độ phức tạp của mô hình tăng lên, nhu cầu về dữ liệu có nhãn cũng tăng theo, khiến cách tiền huấn luyện này gặp khó khăn khi không có sẵn các bộ dữ liệu có nhãn quy mô lớn.

Cách tiếp cận thứ ba là **học tự giám sát** (*self-supervised learning*). Trong mô thức này, mạng nơ-ron được huấn luyện bằng các tín hiệu giám sát rút ra từ chính dữ liệu, chứ không phải từ nhãn do con người gán. Điều này thường được thực hiện bằng cách xây dựng các nhiệm vụ huấn luyện trực tiếp từ dữ liệu chưa gán nhãn, chẳng hạn dự đoán những phần bị thiếu của đầu vào. Mặc dù học tự giám sát gần đây đã trở thành phương pháp chủ đạo trong NLP, triết lý nền tảng của nó có nhiều điểm tương đồng với các khái niệm học máy cũ hơn như **tự huấn luyện** (*self-training*). Trong tự huấn luyện, mô hình cần một lượng nhỏ dữ liệu hạt giống (*seed data*) để

xây dựng một bộ phân loại (*classifier*) ban đầu, rồi bộ phân loại này sinh ra các nhãn giả (*pseudo-label*) cho dữ liệu chưa gán nhãn. Các nhãn giả này sau đó được dùng để tinh chỉnh dần và tự nâng cấp (*bootstrap*) mô hình qua nhiều vòng lặp. Phương pháp này đã được áp dụng thành công cho các lĩnh vực NLP như khử nhập nhằng nghĩa của từ (*word sense disambiguation*) [Yarowsky, 1995] và phân loại văn bản [Blum và Mitchell, 1998]. Tuy nhiên, khác với tự huấn luyện tiêu chuẩn, tiền huấn luyện tự giám sát trong NLP không dựa vào một mô hình ban đầu để gán các nhãn giả rời rạc. Thay vào đó, tín hiệu giám sát được rút ra trực tiếp từ văn bản, và toàn bộ mô hình được huấn luyện từ đầu. Một ví dụ nổi tiếng là huấn luyện các mô hình chuỗi bằng cách che một số token trong văn bản và yêu cầu mô hình dự đoán chúng dựa trên ngữ cảnh xung quanh. Điều này cho phép học tự giám sát quy mô lớn cho các mạng nơ-ron sâu, dẫn tới thành công của tiền huấn luyện trong nhiều nhiệm vụ hiểu, sinh và suy luận (*reasoning*).

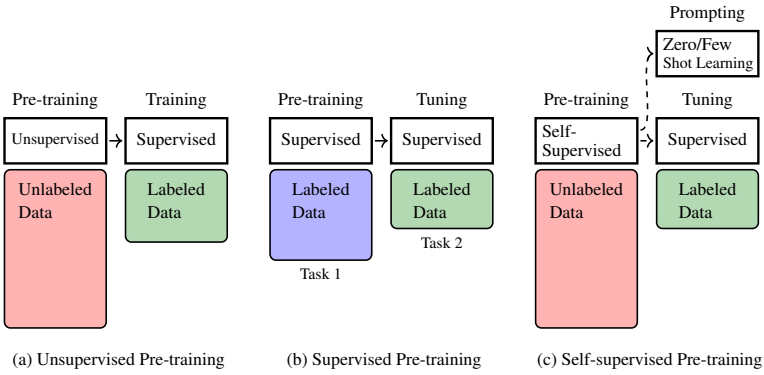
Chú giải của người dịch: Ba kiểu học: so sánh nhanh

- *Có giám sát*: mỗi ví dụ có "đáp án" do người ghi. Ví dụ: câu "Món ăn ngon tuyệt" kèm nhãn *tích cực*.
- *Không giám sát*: chỉ có dữ liệu, không có đáp án; mô hình tự tìm cấu trúc (ví dụ học cách nén rồi tái tạo lại đầu vào).
- *Tự giám sát*: cũng không cần người ghi nhãn, nhưng ta *tự tạo ra đáp án từ chính dữ liệu*. Ví dụ, từ câu "Hôm nay trời rất đẹp", ta che đi một từ: "Hôm nay trời rất ___" và bắt mô hình đoán; đáp án "đẹp" có sẵn trong câu gốc. Vì văn bản trên mạng là vô tận, ta có thể tạo ra hàng tỉ "bài tập" như vậy mà không tốn công gán nhãn.

Tự huấn luyện thì khác: một bộ phân loại nhỏ (học từ ít dữ liệu có nhãn) tự "chấm" nhãn cho dữ liệu chưa có nhãn, rồi dùng chính các nhãn tự chấm này (nhãn giả) để học tiếp. "Cực tiểu địa phương" là điểm thấp của hàm mất mát chỉ trong một vùng lân cận (như đáy một thung lũng nhỏ, chưa chắc là đáy sâu nhất của cả dãy núi).

Hình 7.1 so sánh ba cách tiếp cận tiền huấn luyện nói trên. Tiền huấn luyện tự giám sát thành công đến mức hầu hết các mô hình NLP tiên tiến nhất hiện nay đều dựa trên mô thức này. Vì vậy, trong suốt chương này và phần còn lại của sách, chúng ta sẽ tập trung vào tiền huấn luyện tự giám sát. Ta sẽ thấy các mô hình chuỗi được tiền huấn luyện thông qua tự giám

sát như thế nào và các mô hình đã tiền huấn luyện được áp dụng ra sao.



(a) Unsupervised Pre-training (b) Supervised Pre-training (c) Self-supervised Pre-training

Hình 7.1: Minh họa tiền huấn luyện không giám sát, có giám sát và tự giám sát. Trong tiền huấn luyện không giám sát, việc tiền huấn luyện được thực hiện trên dữ liệu chưa gán nhãn quy mô lớn. Nó có thể được xem như một bước sơ bộ để có một điểm xuất phát tốt cho quá trình tối ưu hóa tiếp theo, dù vẫn cần nhiều công sức để huấn luyện thêm mô hình bằng dữ liệu có nhãn sau khi tiền huấn luyện. Trong tiền huấn luyện có giám sát, giả định nền tảng là các nhiệm vụ học (có giám sát) khác nhau có liên quan với nhau. Vì vậy, ta có thể huấn luyện mô hình trên một nhiệm vụ trước, rồi chuyển mô hình thu được sang một nhiệm vụ khác với một chút công sức huấn luyện hoặc tinh chỉnh. Trong tiền huấn luyện tự giám sát, mô hình được tiền huấn luyện trên dữ liệu chưa gán nhãn quy mô lớn thông qua tự giám sát. Mô hình có thể được huấn luyện hiệu quả theo cách này, và ta có thể thích nghi nó một cách hiệu quả cho các nhiệm vụ mới thông qua tinh chỉnh hoặc prompting.

Giải nghĩa nhãn trong hình: Pre-training = tiền huấn luyện; Training = huấn luyện; Tuning = tinh chỉnh; Unsupervised/Supervised/Self-Supervised = không giám sát/có giám sát/tự giám sát; Unlabeled/Labeled Data = dữ liệu chưa gán nhãn/có nhãn; Task 1/2 = nhiệm vụ 1/2; Prompting – Zero/Few Shot Learning = dùng prompt – học zero-shot/few-shot.

7.1.2 Thích nghi các mô hình đã tiền huấn luyện

Như đã nói ở trên, có hai loại mô hình chính được sử dụng rộng rãi trong tiền huấn luyện NLP.

- **Mô hình mã hóa chuỗi** (*Sequence Encoding Models*). Cho một chuỗi từ hoặc token, mô hình mã hóa chuỗi biểu diễn chuỗi này dưới dạng một vectơ số thực hoặc một chuỗi các vectơ, và thu được một biểu diễn của chuỗi. Biểu diễn này thường được dùng làm đầu vào cho một mô hình khác, chẳng hạn một hệ thống phân loại câu.

- **Mô hình sinh chuỗi** (*Sequence Generation Models*). Trong NLP, sinh chuỗi thường chỉ bài toán sinh ra một chuỗi token dựa trên một ngữ cảnh (*context*) cho trước. Thuật ngữ *ngữ cảnh* mang nghĩa khác nhau tùy ứng dụng. Ví dụ, trong mô hình hóa ngôn ngữ, nó là các token đứng trước; còn trong dịch máy (*machine translation*), nó là chuỗi ở ngôn ngữ nguồn[2].

Ta cần những kỹ thuật khác nhau để áp dụng các mô hình này cho các nhiệm vụ hạ nguồn sau khi tiền huấn luyện. Ở đây ta quan tâm đến hai phương pháp sau.

1. Tinh chỉnh các mô hình đã tiền huấn luyện

Với tiền huấn luyện mã hóa chuỗi, một phương pháp phổ biến để thích nghi các mô hình đã tiền huấn luyện là tinh chỉnh. Gọi $\text{Encode}_\theta(\cdot)$ là một bộ mã hóa (*encoder*) với tham số θ; ví dụ, $\text{Encode}_\theta(\cdot)$ có thể là một bộ mã hóa Transformer tiêu chuẩn. Giả sử ta đã tiền huấn luyện mô hình này theo cách nào đó và thu được các tham số tối ưu $\hat{\theta}$, ta có thể dùng nó để mã hóa một chuỗi đầu vào như sau

$$\mathbf{H} = \text{Encode}_{\hat{\theta}}(\mathbf{x}), \tag{7.2}$$

trong đó $\mathbf{x}$ là chuỗi đầu vào $\{x_0, \cdots, x_m\}$, và $\mathbf{H}$ là biểu diễn đầu ra, là một chuỗi các vectơ số thực $\{\mathbf{h}_0, \mathbf{h}_1, \cdots, \mathbf{h}_m\}$. Vì bộ mã hóa không hoạt động như một hệ thống NLP độc lập, nó thường được tích hợp như một thành phần trong một hệ thống lớn hơn. Ví dụ, xét bài toán phân loại văn bản, trong đó ta xác định cực tính (*polarity*) (chẳng hạn tích cực, tiêu cực hay trung tính) của một văn bản cho trước. Ta có thể xây dựng một hệ thống phân loại văn bản bằng cách đặt một bộ phân loại lên trên bộ mã hóa. Gọi $\text{Classify}_\omega(\cdot)$ là một mạng nơ-ron với tham số ω. Khi đó, mô hình

[2]Chính xác hơn, trong giải mã tự hồi quy (*autoregressive decoding*) của dịch máy, mỗi token ở ngôn ngữ đích được sinh ra dựa trên cả các token đứng trước nó lẫn chuỗi ngôn ngữ nguồn.

phân loại văn bản có thể được biểu diễn dưới dạng

$$\begin{aligned}\Pr_{\omega,\hat{\theta}}(\cdot|\mathbf{x}) &= \text{Classify}_{\omega}(\mathbf{H}) \\ &= \text{Classify}_{\omega}(\text{Encode}_{\hat{\theta}}(\mathbf{x})).\end{aligned} \tag{7.3}$$

Ở đây $\Pr_{\omega,\hat{\theta}}(\cdot|\mathbf{x})$ là một phân phối xác suất trên tập nhãn {positive, negative, neutral} (tích cực, tiêu cực, trung tính), và nhãn có xác suất cao nhất trong phân phối này được chọn làm đầu ra. Để ký hiệu gọn gàng, ta sẽ dùng $F_{\omega,\hat{\theta}}(\cdot)$ để ký hiệu $\text{Classify}_{\omega}(\text{Encode}_{\hat{\theta}}(\cdot))$.

Vì các tham số mô hình ω và $\hat{\theta}$ không được tối ưu hóa tường minh cho nhiệm vụ phân loại đích, ta không thể dùng trực tiếp mô hình kết hợp này. Thay vào đó, ta phải dùng một phiên bản đã được sửa đổi của mô hình, được thích nghi cho nhiệm vụ. Một cách tiếp cận điển hình là tinh chỉnh mô hình bằng cách cung cấp các nhãn tường minh từ nhiệm vụ hạ nguồn. Ta có thể huấn luyện $F_{\omega,\hat{\theta}}(\cdot)$ trên một bộ dữ liệu có nhãn, coi đó như một nhiệm vụ học có giám sát tiêu chuẩn. Quá trình tinh chỉnh này cho ra các tham số được tối ưu thêm, $\tilde{\omega}$ và $\tilde{\theta}$. Một cách khác là ta có thể đóng băng (*freeze*) các tham số bộ mã hóa $\hat{\theta}$ để giữ nguyên trạng thái đã tiền huấn luyện và chỉ tập trung tối ưu hóa ω. Điều này cho phép bộ phân loại được thích nghi một cách hiệu quả để làm việc tốt với bộ mã hóa đã tiền huấn luyện.

Chú giải của người dịch: Các dấu mũ ˆ và dấu ngã ˜

Sách dùng các ký hiệu để phân biệt các "phiên bản" của tham số: θ là tham số nói chung (chưa biết giá trị); $\hat{\theta}$ (đọc "theta mũ") là giá trị *thu được sau tiền huấn luyện*; $\tilde{\theta}$ ("theta ngã") là giá trị *sau khi tinh chỉnh thêm*. Tương tự với ω của bộ phân loại. Hình dung hệ thống gồm hai khối nối tiếp: khối Encode (đã học sẵn, "đọc hiểu" văn bản thành dãy vectơ $\mathbf{H}$) và khối Classify (mới, nhỏ, "ra quyết định" nhãn). Tinh chỉnh có hai lựa chọn: sửa cả hai khối (ω và θ đều thay đổi), hoặc "đóng băng" khối Encode và chỉ huấn luyện khối Classify — rẻ hơn nhiều vì số tham số cần học ít.

Khi đã có mô hình được tinh chỉnh, ta có thể dùng nó để phân loại một văn bản mới. Ví dụ, giả sử có một bình luận được đăng trên một trang web du lịch:

I love the food here. It's amazing!
(Tôi thích đồ ăn ở đây. Thật tuyệt vời!)

Đầu tiên, ta tách văn bản này thành các token[3], rồi đưa chuỗi token $\mathbf{x}_{\text{new}}$ vào mô hình đã tinh chỉnh $F_{\tilde{\omega},\tilde{\theta}}(\cdot)$. Mô hình sinh ra một phân phối trên các lớp như sau

$$F_{\tilde{\omega},\tilde{\theta}}(\mathbf{x}_{\text{new}}) = \begin{bmatrix} \Pr(\text{positive}|\mathbf{x}_{\text{new}}) & \Pr(\text{negative}|\mathbf{x}_{\text{new}}) & \Pr(\text{neutral}|\mathbf{x}_{\text{new}}) \end{bmatrix}. \tag{7.4}$$

Sau đó ta chọn nhãn có xác suất cao nhất làm đầu ra. Trong ví dụ này, nhãn đó là positive (tích cực).

Chú giải của người dịch: Ví dụ bằng số

Giả sử mô hình trả về vectơ $[0.92 \quad 0.03 \quad 0.05]$. Ba số này là xác suất của ba nhãn tích cực, tiêu cực, trung tính, và có tổng bằng 1. Số lớn nhất là 0.92 ứng với *tích cực*, nên mô hình kết luận bình luận mang cảm xúc tích cực. Ký hiệu $\Pr(\text{positive}|\mathbf{x}_{\text{new}})$ đọc là "xác suất nhãn tích cực khi biết văn bản $\mathbf{x}_{\text{new}}$" (xác suất có điều kiện).

Nhìn chung, lượng dữ liệu có nhãn dùng trong tinh chỉnh nhỏ so với lượng dữ liệu tiền huấn luyện, nên tinh chỉnh tốn ít chi phí tính toán hơn. Điều này khiến việc thích nghi các mô hình đã tiền huấn luyện trở nên rất hiệu quả trong thực tế: với một mô hình đã tiền huấn luyện và một nhiệm vụ hạ nguồn, ta chỉ cần thu thập một ít dữ liệu có nhãn và điều chỉnh nhẹ các tham số mô hình trên dữ liệu này. Phần thảo luận chi tiết hơn về tinh chỉnh có thể xem ở Mục 7.4.

2. Prompting các mô hình đã tiền huấn luyện

Khác với các mô hình mã hóa chuỗi, các mô hình sinh chuỗi thường được dùng độc lập để giải quyết các bài toán sinh ngôn ngữ, chẳng hạn hỏi đáp (*question answering*) và dịch máy, mà không cần thêm mô-đun nào. Do đó, việc tinh chỉnh các mô hình này như những hệ thống hoàn chỉnh trên các nhiệm vụ hạ nguồn là điều đơn giản. Ví dụ, ta có thể tinh chỉnh một mô hình đa ngôn ngữ dạng bộ mã hóa–bộ giải mã (*encoder-decoder*) đã tiền huấn luyện trên một ít dữ liệu song ngữ để cải thiện hiệu năng của nó trên một nhiệm vụ dịch cụ thể.

Trong số các mô hình sinh chuỗi, một ví dụ đáng chú ý là các mô hình

[3]Văn bản có thể được tách token theo nhiều cách khác nhau. Một trong những cách đơn giản nhất là tách văn bản thành các từ tiếng Anh và dấu câu {I, love, the, food, here, ., It, 's, amazing, !}.

ngôn ngữ lớn được huấn luyện trên lượng dữ liệu rất lớn. Các mô hình này được huấn luyện đơn giản chỉ để dự đoán token tiếp theo khi biết các token đứng trước. Mặc dù dự đoán token là một nhiệm vụ đơn giản vốn được dùng từ lâu cho mô hình hóa ngôn ngữ, việc thực hiện nó ở quy mô lớn giúp mô hình thu nhận được tri thức rộng về ngôn ngữ và về thế giới. Kết quả là các LLM đã tiền huấn luyện thể hiện khả năng dự đoán token đáng kinh ngạc, khiến ta có thể đưa nhiều nhiệm vụ NLP về dạng những bài toán sinh văn bản đơn giản thông qua prompting. Ví dụ, ta có thể đưa bài toán phân loại văn bản ở trên về dạng một nhiệm vụ sinh văn bản.

I love the food here. It's amazing! I'm ___
(Tôi thích đồ ăn ở đây. Thật tuyệt vời! Tôi thấy ___)

Ở đây __ chỉ từ hoặc cụm từ mà ta muốn dự đoán (gọi là **phần hoàn thành** (*completion*)). Nếu từ được dự đoán là *happy* (vui), *glad* (vui mừng), *satisfied* (hài lòng), hoặc một từ tích cực liên quan, ta có thể phân loại văn bản là positive. Ví dụ này cho thấy một phương pháp prompting đơn giản, trong đó ta nối văn bản đầu vào với *I'm* ("Tôi thấy") để tạo thành một prompt (*prompt*). Sau đó, phần hoàn thành giúp quyết định nhãn nào được gán cho văn bản gốc.

Với khả năng hiểu và sinh ngôn ngữ mạnh mẽ của các mô hình ngôn ngữ lớn, một prompt có thể chỉ dẫn các mô hình này thực hiện những nhiệm vụ phức tạp hơn. Ví dụ sau minh họa cách chỉ dẫn một LLM thực hiện phân loại cực tính:

> Giả sử cực tính của một văn bản là một nhãn được chọn từ {tích cực, tiêu cực, trung tính}. Hãy xác định cực tính của đầu vào.
> Đầu vào: Tôi thích đồ ăn ở đây. Thật tuyệt vời!
> Cực tính: ___

Hai câu đầu tiên là phần mô tả nhiệm vụ. Đầu vào (*Input*) và Cực tính (*Polarity*) lần lượt là các dấu hiệu chỉ đầu vào và đầu ra. Ta kỳ vọng mô hình sẽ hoàn thành văn bản và đồng thời đưa ra nhãn cực tính đúng. Bằng cách dùng các prompt dựa trên chỉ dẫn, ta có thể thích nghi các mô hình ngôn ngữ lớn để giải các bài toán NLP mà không cần huấn luyện thêm.

Ví dụ này cũng cho thấy khả năng học zero-shot (*zero-shot learning*) của các mô hình ngôn ngữ lớn: chúng có thể thực hiện những nhiệm vụ chưa từng gặp trong quá trình huấn luyện. Một phương pháp khác để

khơi gợi những năng lực mới từ mạng nơ-ron là học few-shot (*few-shot learning*). Điều này thường được thực hiện thông qua **học trong ngữ cảnh** (*in-context learning*) (**ICL**). Cụ thể hơn, ta thêm vào một số mẫu minh họa cách một đầu vào tương ứng với một đầu ra. Các mẫu này, được gọi là **ví dụ minh họa** (*demonstrations*), được dùng để dạy các mô hình ngôn ngữ lớn cách thực hiện nhiệm vụ. Dưới đây là một ví dụ có kèm các ví dụ minh họa

Giả sử cực tính của một văn bản là một nhãn được chọn từ {tích cực, tiêu cực, trung tính}. Hãy xác định cực tính của đầu vào.
Đầu vào: Giao thông vào giờ cao điểm thật tồi tệ, khiến việc đến sân bay đúng giờ trở nên khó khăn.
Cực tính: Tiêu cực
Đầu vào: Thời tiết ở đây thật tuyệt.
Cực tính: Tích cực
Đầu vào: Tôi thích đồ ăn ở đây. Thật tuyệt vời!
Cực tính: ___

Chú giải của người dịch: Zero-shot và few-shot

"Shot" ở đây có thể hiểu là "ví dụ mẫu". *Zero-shot*: không cho mô hình ví dụ mẫu nào, chỉ mô tả nhiệm vụ (prompt thứ nhất). *Few-shot*: cho vài ví dụ mẫu đã có đáp án (prompt thứ hai có 2 ví dụ, gọi là 2-shot). Điều đáng chú ý là trong cả hai trường hợp, tham số của mô hình *không hề thay đổi*: mô hình chỉ đọc prompt và viết tiếp. Các ví dụ minh họa giúp mô hình hiểu rõ hơn định dạng và ý nghĩa của nhãn cần đưa ra.

Prompting và học trong ngữ cảnh đóng vai trò quan trọng trong sự trỗi dậy gần đây của các mô hình ngôn ngữ lớn. Chúng ta sẽ bàn chi tiết hơn về các kỹ thuật này ở Chương 9. Tuy nhiên, cần lưu ý rằng mặc dù prompting là một cách mạnh mẽ để thích nghi các mô hình ngôn ngữ lớn, vẫn cần một số công sức tinh chỉnh để bảo đảm mô hình có thể làm theo chỉ dẫn một cách chính xác. Ngoài ra, quá trình tinh chỉnh rất quan trọng để căn chỉnh (*alignment*) các giá trị của những mô hình này với giá trị của con người. Phần thảo luận chi tiết hơn về tinh chỉnh có thể xem ở Chương 10.

7.2 Các nhiệm vụ tiền huấn luyện tự giám sát

Trong mục này, ta xem xét các cách tiếp cận tiền huấn luyện tự giám sát cho các kiến trúc nơ-ron khác nhau, bao gồm kiến trúc chỉ có bộ giải mã (*decoder-only*), chỉ có bộ mã hóa (*encoder-only*) và kiến trúc bộ mã hóa–bộ giải mã. Ta giới hạn thảo luận ở Transformer vì chúng là nền tảng của hầu hết các mô hình đã tiền huấn luyện trong NLP. Tuy nhiên, tiền huấn luyện là một khái niệm rộng, nên ở đây chúng tôi chỉ giới thiệu ngắn gọn các cách tiếp cận cơ bản để mục này được súc tích.

7.2.1 Tiền huấn luyện chỉ với bộ giải mã

Kiến trúc chỉ có bộ giải mã đã được dùng rộng rãi để phát triển các mô hình ngôn ngữ [Radford và cộng sự, 2018]. Ví dụ, ta có thể dùng một bộ giải mã (*decoder*) Transformer làm mô hình ngôn ngữ bằng cách đơn giản là bỏ đi các tầng con chú ý chéo (*cross-attention*) của nó. Một mô hình như vậy dự đoán phân phối xác suất trên bộ từ vựng cho token tiếp theo dựa trên các token đứng trước, và đầu ra dự đoán là token có xác suất cao nhất. Cách tiêu chuẩn để huấn luyện mô hình này, giống như trong mô hình hóa ngôn ngữ truyền thống, là cực tiểu hóa một hàm mất mát (*loss function*) trên một tập các chuỗi token.

Gọi $\text{Decoder}_\theta(\cdot)$ là một bộ giải mã với tham số θ. Tại mỗi vị trí i, bộ giải mã sinh ra một phân phối cho token tiếp theo dựa trên ngữ cảnh đứng trước $\{x_0, \ldots, x_i\}$, ký hiệu là $\Pr_\theta(\cdot|x_0, \ldots, x_i)$ (hay viết tắt là $\mathbf{p}^\theta_{i+1}$). Giả sử ta có phân phối đích chuẩn (*gold-standard*) tại cùng vị trí, ký hiệu là $\mathbf{p}^{\text{gold}}_{i+1}$. Với mô hình hóa ngôn ngữ, ta có thể xem $\mathbf{p}^{\text{gold}}_{i+1}$ là biểu diễn one-hot (*one-hot representation*) của từ tiếp theo đúng. Sau đó ta định nghĩa một hàm mất mát $\mathcal{L}(\mathbf{p}^\theta_{i+1}, \mathbf{p}^{\text{gold}}_{i+1})$ để đo độ sai khác giữa dự đoán của mô hình và phân phối đích. Trong NLP, hàm mất mát entropy chéo thường được sử dụng.

Cho một chuỗi token $\mathbf{x} = \{x_0, \cdots, x_m\}$, mất mát trên chuỗi này là tổng mất mát tại các vị trí $\{0, \cdots, m-1\}$, cho bởi

$$\text{Loss}_\theta(x_0, \cdots, x_m) = \sum_{i=0}^{m-1} \mathcal{L}(\mathbf{p}^\theta_{i+1}, \mathbf{p}^{\text{gold}}_{i+1})$$

$$= \sum_{i=0}^{m-1} \text{CrossEntropy}(\mathbf{p}_{i+1}^{\theta}, \mathbf{p}_{i+1}^{\text{gold}}), \tag{7.5}$$

trong đó $\text{CrossEntropy}(\cdot)$ ký hiệu hàm mất mát entropy chéo tiêu chuẩn, và $\mathbf{p}_{i+1}^{\text{gold}}$ là biểu diễn one-hot của x_{i+1}.

Chú giải của người dịch: One-hot và entropy chéo – ví dụ nhỏ

Giả sử bộ từ vựng chỉ có 4 từ: {tôi, thích, mèo, chó} và từ đúng tiếp theo là "mèo". Biểu diễn one-hot của "mèo" là $[0\ 0\ 1\ 0]$: chỉ có một số 1 ở đúng vị trí của từ đó. Nếu mô hình dự đoán $\mathbf{p}^{\theta} = [0.1 \quad 0.1 \quad 0.6 \quad 0.2]$, thì entropy chéo giữa hai phân phối bằng $-\log(\text{xác suất mô hình gán cho từ đúng}) = -\log 0.6 \approx 0.51$ (logarit tự nhiên). Nếu mô hình tự tin hơn, gán 0.9 cho "mèo", mất mát giảm còn $-\log 0.9 \approx 0.11$; nếu gán chỉ 0.01, mất mát tăng vọt tới ≈ 4.6. Vì vậy, cực tiểu hóa entropy chéo nghĩa là ép mô hình gán xác suất cao cho từ đúng. Phương trình (7.5) chỉ đơn giản cộng các mất mát này qua mọi vị trí trong câu.

Hàm mất mát này có thể được mở rộng cho một tập các chuỗi $\mathcal{D}$. Khi đó, mục tiêu của tiền huấn luyện là tìm các tham số tối ưu cực tiểu hóa mất mát trên $\mathcal{D}$

$$\hat{\theta} = \arg\min_{\theta} \sum_{\mathbf{x} \in \mathcal{D}} \text{Loss}_{\theta}(\mathbf{x}). \tag{7.6}$$

Lưu ý rằng mục tiêu này tương đương về mặt toán học với ước lượng hợp lý cực đại (*maximum likelihood estimation*), và có thể được viết lại như sau:

$$\begin{aligned} \hat{\theta} &= \arg\max_{\theta} \sum_{\mathbf{x} \in \mathcal{D}} \log \Pr_{\theta}(\mathbf{x}) \\ &= \arg\max_{\theta} \sum_{\mathbf{x} \in \mathcal{D}} \sum_{i=0}^{|\mathbf{x}|-2} \log \Pr_{\theta}(x_{i+1} | x_0, \ldots, x_i). \end{aligned} \tag{7.7}$$

Với các tham số đã tối ưu $\hat{\theta}$ này, ta có thể dùng mô hình ngôn ngữ đã tiền huấn luyện $\text{Decoder}_{\hat{\theta}}(\cdot)$ để tính xác suất $\Pr_{\hat{\theta}}(x_{i+1} | x_0, \cdots, x_i)$ tại mỗi vị trí của một chuỗi cho trước.

Chú giải của người dịch: Đọc arg min, arg max **và vì sao (7.6) tương đương (7.7)**

arg $\min_\theta f(\theta)$ là "giá trị của θ làm f nhỏ nhất" (khác với min f là bản thân giá trị nhỏ nhất). Ví dụ với $f(\theta) = (\theta-3)^2+1$ thì $\min f = 1$ còn $\arg\min f = 3$. Tương tự, arg max là giá trị làm hàm lớn nhất.

Vì mất mát entropy chéo với nhãn one-hot đúng bằng $-\log$ của xác suất gán cho token đúng, nên tổng mất mát bằng $-\sum \log \Pr_\theta(x_{i+1}|x_0,\ldots,x_i)$. Làm cực tiểu một số âm của một đại lượng cũng chính là làm cực đại đại lượng đó, nên arg min trong (7.6) trở thành arg max trong (7.7). Thêm nữa, theo quy tắc nhân xác suất, xác suất cả câu bằng tích các xác suất có điều kiện của từng token (khi biết các token trước), và logarit biến tích thành tổng: $\log(ab) = \log a + \log b$ — đó là lý do có dấu $\sum_i$ bên trong. Chỉ số chạy tới $|\mathbf{x}| - 2$ vì chuỗi có $|\mathbf{x}| = m + 1$ token (x_0 đến x_m) và ta dự đoán từ x_1 đến x_m.

7.2.2 Tiền huấn luyện chỉ với bộ mã hóa

Như đã định nghĩa ở Mục 7.1.2, bộ mã hóa $\text{Encode}_\theta(\cdot)$ là một hàm đọc một chuỗi token $\mathbf{x} = \{x_0,\ldots,x_m\}$ và tạo ra một chuỗi vectơ $\mathbf{H} = \{\mathbf{h}_0,\ldots,\mathbf{h}_m\}$[4]. Việc huấn luyện mô hình này không đơn giản, vì ta không có dữ liệu chuẩn để đo chất lượng của các biểu diễn liên tục mà hàm này tạo ra. Một cách tiếp cận điển hình cho tiền huấn luyện bộ mã hóa là kết hợp bộ mã hóa với một tầng đầu ra dành riêng cho nhiệm vụ để có được những tín hiệu giám sát sẵn có. Hình 7.2 cho thấy một kiến trúc phổ biến để tiền huấn luyện bộ mã hóa Transformer, trong đó ta thêm một tầng Softmax lên trên bộ mã hóa. Kiến trúc này về cơ bản giống với kiến trúc của mô hình ngôn ngữ dựa trên bộ giải mã, và đầu ra là một chuỗi các

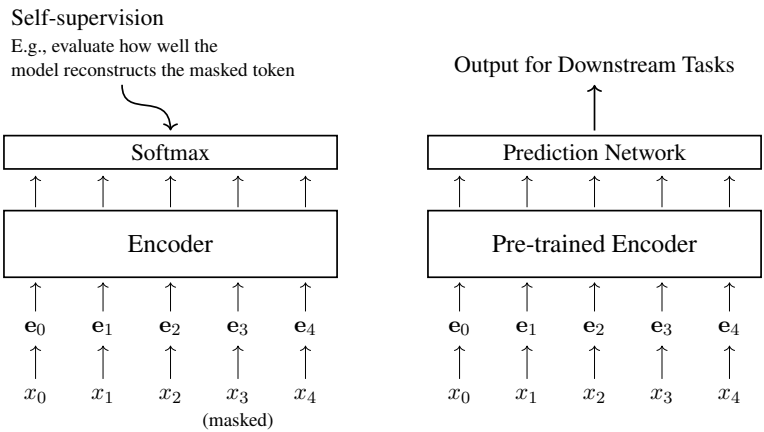
[4]Nếu ta xem $\mathbf{h}_i$ là một vectơ hàng, $\mathbf{H}$ có thể được viết là

$$\mathbf{H} = \begin{bmatrix} \mathbf{h}_0 \\ \vdots \\ \mathbf{h}_m \end{bmatrix}.$$

phân phối xác suất:

$$\begin{bmatrix} \mathbf{p}_1^{\mathbf{W},\theta} \\ \vdots \\ \mathbf{p}_m^{\mathbf{W},\theta} \end{bmatrix} = \text{Softmax}_{\mathbf{W}}(\text{Encode}_\theta(\mathbf{x})). \tag{7.8}$$

Ở đây $\mathbf{p}_i^{\mathbf{W},\theta}$ là phân phối đầu ra $\Pr(\cdot|\mathbf{x})$ tại vị trí i. Ta dùng $\text{Softmax}_{\mathbf{W}}(\cdot)$ để chỉ rằng tầng Softmax được tham số hóa bởi $\mathbf{W}$, tức là $\text{Softmax}_{\mathbf{W}}(\mathbf{H}) = \text{Softmax}(\mathbf{H} \cdot \mathbf{W})$. Để ký hiệu đơn giản, đôi khi ta sẽ bỏ các chỉ số trên $\mathbf{W}$ và θ gắn với mỗi phân phối xác suất.



(a) Pre-training (b) Applying the Pre-trained Encoder

Hình 7.2: Tiền huấn luyện một bộ mã hóa Transformer (trái) rồi áp dụng bộ mã hóa đã tiền huấn luyện (phải). Trong giai đoạn tiền huấn luyện, bộ mã hóa cùng với một tầng Softmax được huấn luyện thông qua tự giám sát. Trong giai đoạn áp dụng, tầng Softmax bị bỏ đi, và bộ mã hóa đã tiền huấn luyện được kết hợp với một mạng dự đoán để giải quyết các bài toán cụ thể. Nhìn chung, để thích nghi tốt hơn với các nhiệm vụ này, hệ thống được tinh chỉnh bằng dữ liệu có nhãn.

Giải nghĩa nhãn trong hình: Self-supervision = tự giám sát (ví dụ: đánh giá mô hình tái tạo token bị che tốt đến đâu); Encoder / Pre-trained Encoder = bộ mã hóa / bộ mã hóa đã tiền huấn luyện; (masked) = bị che; Prediction Network = mạng dự đoán; Output for Downstream Tasks = đầu ra cho các nhiệm vụ hạ nguồn; $\mathbf{e}_i$ là vectơ nhúng của token x_i.

Điểm khác biệt giữa mô hình này và các mô hình ngôn ngữ tiêu chuẩn là đầu ra $\mathbf{p}_i$ mang ý nghĩa khác nhau trong tiền huấn luyện bộ mã hóa và trong mô hình hóa ngôn ngữ. Trong mô hình hóa ngôn ngữ, $\mathbf{p}_i$ là phân phối xác suất để dự đoán token tiếp theo. Điều này tuân theo quá trình giải mã tự hồi quy: mô hình ngôn ngữ chỉ quan sát các từ cho đến vị trí i và dự đoán từ tiếp theo. Ngược lại, trong tiền huấn luyện bộ mã hóa, toàn bộ chuỗi được quan sát cùng lúc, và do đó việc dự đoán một token là quá dễ khi mô hình đã nhìn thấy token đó.

Chú giải của người dịch: Softmax và "nhìn trộm đáp án"

Hàm Softmax biến một dãy số thực bất kỳ thành các xác suất dương có tổng bằng 1: $\text{Softmax}(z_1, \ldots, z_n)_k = e^{z_k} / \sum_j e^{z_j}$. Ví dụ $(2, 1, 0) \mapsto (0.67,\ 0.24,\ 0.09)$. Ma trận $\mathbf{W}$ chuyển mỗi vectơ $\mathbf{h}_i$ thành một điểm số cho từng từ trong bộ từ vựng trước khi đưa qua Softmax.

Vấn đề nêu ở đoạn trên: bộ mã hóa "nhìn" được toàn bộ câu, kể cả từ mà nó cần đoán. Nếu bắt nó đoán từ ở vị trí i mà từ đó vẫn nằm sẵn trong đầu vào thì nó chỉ việc "chép" lại — giống như làm bài khi đáp án in ngay trong đề. Vì thế cần một cách đặt bài toán khác, chẳng hạn *che* từ cần đoán đi, như mục tiếp theo trình bày.

1. Mô hình hóa ngôn ngữ có che

Một trong những phương pháp tiền huấn luyện bộ mã hóa phổ biến nhất là **mô hình hóa ngôn ngữ có che** (*masked language modeling*), là nền tảng của mô hình BERT nổi tiếng [Devlin và cộng sự, 2019]. Ý tưởng của mô hình hóa ngôn ngữ có che là tạo ra các thử thách dự đoán bằng cách che đi một số token trong chuỗi đầu vào và huấn luyện mô hình dự đoán các token bị che. Theo nghĩa này, bài toán mô hình hóa ngôn ngữ thông thường, đôi khi được gọi là **mô hình hóa ngôn ngữ nhân quả** (*causal language modeling*), là một trường hợp đặc biệt của mô hình hóa ngôn ngữ có che: tại mỗi vị trí, ta che các token trong ngữ cảnh bên phải, và dự đoán token ở vị trí này bằng ngữ cảnh bên trái của nó. Tuy nhiên, trong mô hình hóa ngôn ngữ nhân quả, chỉ ngữ cảnh bên trái được dùng để dự đoán token, và mô hình không được truy cập các token trong ngữ cảnh bên phải. Ngược lại, trong mô hình hóa ngôn ngữ có che, tất cả các token không bị che đều được dùng để dự đoán từ, dẫn tới một mô hình hai chiều (*bidirectional*) đưa ra dự đoán dựa trên cả ngữ cảnh bên trái lẫn bên phải.

Một cách hình thức hơn, với chuỗi đầu vào $\mathbf{x} = x_0 \cdots x_m$, giả sử ta che các token tại các vị trí $\mathcal{A}(\mathbf{x}) = \{i_1, \cdots, i_u\}$. Khi đó ta thu được một chuỗi token bị che $\bar{\mathbf{x}}$, trong đó token tại mỗi vị trí thuộc $\mathcal{A}(\mathbf{x})$ được thay bằng một ký hiệu đặc biệt [MASK]. Ví dụ, với chuỗi sau

The early bird catches the worm
(nghĩa đen: Con chim dậy sớm bắt được sâu)

ta có thể có một chuỗi token bị che như thế này

The [MASK] bird catches the [MASK]

trong đó ta che các token *early* và *worm* (tức là $i_1 = 2$ và $i_2 = 6$).

Bây giờ ta có hai chuỗi $\mathbf{x}$ và $\bar{\mathbf{x}}$. Mô hình sau đó được tối ưu hóa sao cho ta có thể dự đoán đúng $\mathbf{x}$ dựa trên $\bar{\mathbf{x}}$. Điều này có thể được xem như một quá trình giống tự mã hóa (*autoencoding*), và mục tiêu huấn luyện là cực đại hóa xác suất tái tạo $\Pr(\mathbf{x}|\bar{\mathbf{x}})$. Lưu ý rằng có một sự tương ứng đơn giản theo từng vị trí giữa $\mathbf{x}$ và $\bar{\mathbf{x}}$. Vì một token không bị che trong $\bar{\mathbf{x}}$ giống hệt token trong $\mathbf{x}$ ở cùng vị trí, ta không cần xét việc dự đoán cho token không bị che này. Điều này dẫn tới một mục tiêu huấn luyện đơn giản hơn, chỉ cực đại hóa xác suất cho các token bị che. Ta có thể biểu diễn mục tiêu này theo kiểu ước lượng hợp lý cực đại

$$(\widehat{\mathbf{W}}, \hat{\theta}) = \arg\max_{\mathbf{W},\theta} \sum_{\mathbf{x}\in\mathcal{D}} \sum_{i\in\mathcal{A}(\mathbf{x})} \log \Pr_i^{\mathbf{W},\theta}(x_i|\bar{\mathbf{x}}), \tag{7.9}$$

hoặc biểu diễn nó bằng hàm mất mát entropy chéo

$$(\widehat{\mathbf{W}}, \hat{\theta}) = \arg\min_{\mathbf{W},\theta} \sum_{\mathbf{x}\in\mathcal{D}} \sum_{i\in\mathcal{A}(\mathbf{x})} \text{CrossEntropy}(\mathbf{p}_i^{\mathbf{W},\theta}, \mathbf{p}_i^{\text{gold}}), \tag{7.10}$$

trong đó $\Pr_k^{\mathbf{W},\theta}(x_k|\bar{\mathbf{x}})$ là xác suất của token đúng x_k tại vị trí k khi biết đầu vào đã bị làm hỏng $\bar{\mathbf{x}}$, và $\mathbf{p}_k^{\mathbf{W},\theta}$ là phân phối xác suất tại vị trí k khi biết đầu vào bị làm hỏng $\bar{\mathbf{x}}$. Để minh họa, xét ví dụ trên, trong đó hai token của chuỗi "*the early bird catches the worm*" bị che. Với ví dụ này, mục tiêu là cực đại hóa tổng các log-xác suất

$$\begin{aligned}\text{Loss} = \log \Pr(x_2 = \textit{early}| \\ \bar{\mathbf{x}} = [\text{CLS}]\ \textit{The}\ \underbrace{[\text{MASK}]}_{\bar{x}_2}\ \textit{bird catches the}\ \underbrace{[\text{MASK}]}_{\bar{x}_6}) +\end{aligned}$$

$$\log \Pr(x_6 = \textit{worm} | \bar{\mathbf{x}} = [\text{CLS}]\ \textit{The}\ \underbrace{[\text{MASK}]}_{\bar{x}_2}\ \textit{bird catches the}\ \underbrace{[\text{MASK}]}_{\bar{x}_6}). \tag{7.11}$$

Khi đã thu được các tham số tối ưu $\widehat{\mathbf{W}}$ và $\hat{\theta}$, ta có thể bỏ $\widehat{\mathbf{W}}$. Sau đó, ta có thể tinh chỉnh thêm bộ mã hóa đã tiền huấn luyện $\text{Encode}_{\hat{\theta}}(\cdot)$ hoặc áp dụng trực tiếp nó cho các nhiệm vụ hạ nguồn.

Chú giải của người dịch: Che từ để học – giống bài tập điền từ

Mô hình hóa ngôn ngữ có che giống hệt bài tập "điền vào chỗ trống" trong môn Ngoại ngữ. Với câu "[CLS] The [MASK] bird catches the [MASK]", mô hình phải đoán ô 1 là *early*, ô 2 là *worm*, và được "chấm điểm" bằng log xác suất nó gán cho các đáp án đúng — Phương trình (7.11) chính là tổng hai điểm số đó (hai số hạng ứng với hai ô trống). Lưu ý vị trí đếm từ 0: [CLS] là x_0, *The* là x_1, *early* là x_2, …, *worm* là x_6. Khác với mô hình ngôn ngữ nhân quả (chỉ nhìn bên trái), ở đây mô hình được nhìn cả hai phía: từ "bird" bên phải giúp đoán "early" bên trái.

2. Mô hình hóa ngôn ngữ hoán vị

Mặc dù mô hình hóa ngôn ngữ có che đơn giản và được áp dụng rộng rãi, nó lại phát sinh những vấn đề mới. Một nhược điểm là việc dùng token đặc biệt [MASK], vốn chỉ được dùng trong lúc huấn luyện mà không có lúc kiểm tra. Điều này dẫn tới sự sai lệch giữa huấn luyện và suy luận (*inference*). Hơn nữa, quá trình tự mã hóa bỏ qua sự phụ thuộc giữa các token bị che. Ví dụ, trong ví dụ đã bàn ở trên, việc dự đoán x_2 (tức token bị che thứ nhất) được thực hiện độc lập với x_6 (tức token bị che thứ hai), mặc dù x_6 và x_2 phụ thuộc lẫn nhau trong ngữ cảnh của câu.

Những vấn đề này có thể được giải quyết bằng cách tiếp cận tiền huấn luyện **mô hình hóa ngôn ngữ hoán vị** (*permuted language modeling*) [Yang và cộng sự, 2019]. Tương tự như mô hình hóa ngôn ngữ nhân quả, mô hình hóa ngôn ngữ hoán vị đưa ra các dự đoán token một cách tuần tự. Tuy nhiên, khác với mô hình hóa nhân quả trong đó các dự đoán tuân theo trình tự tự nhiên của văn bản (như từ trái sang phải hoặc từ phải sang trái), mô hình hóa ngôn ngữ hoán vị cho phép dự đoán theo thứ tự bất kỳ. Cách tiếp cận này đơn giản: ta xác định một thứ tự dự đoán

token rồi huấn luyện mô hình theo cách mô hình hóa ngôn ngữ tiêu chuẩn, như đã mô tả ở Mục 7.2.1. Lưu ý rằng trong cách tiếp cận này, thứ tự thực của các token trong văn bản vẫn giữ nguyên, và chỉ thứ tự mà ta dự đoán các token này khác với mô hình hóa ngôn ngữ tiêu chuẩn. Ví dụ, xét một chuỗi gồm 5 token $x_0x_1x_2x_3x_4$. Gọi $\mathbf{e}_i$ là vectơ nhúng của x_i (tức là tổ hợp của nhúng token và nhúng vị trí (*positional embedding*)). Trong mô hình hóa ngôn ngữ tiêu chuẩn, ta sẽ sinh chuỗi này theo thứ tự $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4$. Xác suất của chuỗi có thể được mô hình hóa thông qua một quá trình sinh.

$$\begin{aligned}\Pr(\mathbf{x}) = \; & \Pr(x_0) \cdot \Pr(x_1|x_0) \cdot \Pr(x_2|x_0, x_1) \cdot \Pr(x_3|x_0, x_1, x_2) \cdot \\ & \Pr(x_4|x_0, x_1, x_2, x_3). \end{aligned} \tag{7.12}$$

Trong thực tế, mô hình lấy điều kiện trên các vectơ nhúng token chứ không phải trên chính các token. Do đó, ta có thể viết lại biểu thức trên thành

$$\begin{aligned}\Pr(\mathbf{x}) = \; & \Pr(x_0) \cdot \Pr(x_1|\mathbf{e}_0) \cdot \Pr(x_2|\mathbf{e}_0, \mathbf{e}_1) \cdot \Pr(x_3|\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_2) \cdot \\ & \Pr(x_4|\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3). \end{aligned} \tag{7.13}$$

Bây giờ, hãy xét một thứ tự dự đoán token khác: $x_0 \rightarrow x_4 \rightarrow x_2 \rightarrow x_1 \rightarrow x_3$. Quá trình sinh chuỗi khi đó có thể được biểu diễn như sau:

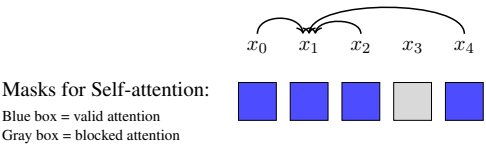
$$\begin{aligned}\Pr(\mathbf{x}) = \; & \Pr(x_0) \cdot \Pr(x_4|\mathbf{e}_0) \cdot \Pr(x_2|\mathbf{e}_0, \mathbf{e}_4) \cdot \Pr(x_1|\mathbf{e}_0, \mathbf{e}_4, \mathbf{e}_2) \cdot \\ & \Pr(x_3|\mathbf{e}_0, \mathbf{e}_4, \mathbf{e}_2, \mathbf{e}_1). \end{aligned} \tag{7.14}$$

Thứ tự dự đoán mới này cho phép một số token được sinh ra dựa trên một ngữ cảnh rộng hơn, thay vì chỉ giới hạn ở các token đứng trước như trong các mô hình ngôn ngữ tiêu chuẩn. Ví dụ, khi sinh x_3, mô hình xét cả ngữ cảnh bên trái (tức $\{\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_2\}$) lẫn ngữ cảnh bên phải (tức $\{\mathbf{e}_4\}$). Các vectơ nhúng $\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_4$ có chứa thông tin vị trí của x_0, x_1, x_2, x_4, nhờ đó giữ được thứ tự ban đầu của các token. Kết quả là cách tiếp cận này phần nào giống với mô hình hóa ngôn ngữ có che: ta che x_3 và dùng các token xung quanh nó là x_0, x_1, x_2, x_4 để dự đoán token này.

Chú giải của người dịch: Quy tắc nhân xác suất và "đổi thứ tự đoán"

Phương trình (7.12) dựa trên quy tắc nhân: $\Pr(A, B) = \Pr(A) \cdot \Pr(B|A)$. Áp dụng liên tiếp, xác suất cả chuỗi bằng tích xác suất của từng token khi biết những token đã "đoán" trước nó. Quy tắc này đúng với *mọi* thứ tự, nên ta có thể đoán theo thứ tự x_0, x_4, x_2, x_1, x_3 như (7.14) mà xác suất cả chuỗi về lý thuyết không đổi. Điểm lợi: khi đoán x_3 ở bước cuối, mô hình đã biết cả x_4 (bên phải) — điều mà mô hình trái-sang-phải không làm được. Vì mỗi vectơ $\mathbf{e}_i$ có "gắn" thông tin vị trí, mô hình vẫn biết x_4 đứng cuối câu dù nó được đoán thứ hai.

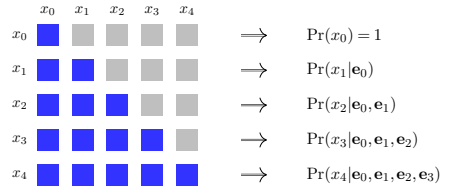
Việc cài đặt các mô hình ngôn ngữ hoán vị tương đối dễ dàng với Transformer. Vì bản thân cơ chế tự chú ý (*self-attention*) là bất biến với hoán vị (*permutation-invariant*), và thông tin vị trí được mã hóa riêng bằng các vectơ nhúng vị trí, ta không cần sắp xếp lại một cách tường minh chuỗi đầu vào để có một phân tích thành tích như Phương trình (7.14). Thay vào đó, việc hoán vị có thể được thực hiện bằng cách đặt các mặt nạ (*mask*) thích hợp cho tự chú ý. Ví dụ, xét bước tính $\Pr(x_1|\mathbf{e}_0, \mathbf{e}_4, \mathbf{e}_2)$. Ta có thể giữ x_0, x_1, x_2, x_3, x_4 ở các vị trí ban đầu của chúng và chỉ cần ngăn x_1 chú ý tới x_3 trong quá trình tính tự chú ý, như minh họa dưới đây



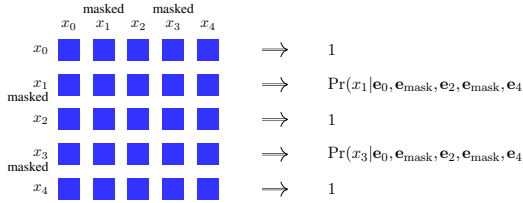
(Masks for Self-attention = mặt nạ cho tự chú ý; Blue box = ô xanh: chú ý hợp lệ; Gray box = ô xám: chú ý bị chặn)

Để minh họa rõ hơn, ta so sánh kết quả đặt mặt nạ tự chú ý của mô hình hóa ngôn ngữ nhân quả, mô hình hóa ngôn ngữ có che và mô hình hóa ngôn ngữ hoán vị trong Hình 7.3.

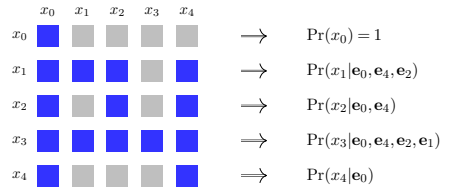
Giải nghĩa nhãn trong hình: (a) Causal Language Modeling = mô hình hóa ngôn ngữ nhân quả; (b) Masked Language Modeling = mô hình hóa ngôn ngữ có che (masked = bị che); (c) Permuted Language Modeling = mô hình hóa ngôn ngữ hoán vị; order = thứ tự dự đoán.



(a) Causal Language Modeling (order: $x_0 \to x_1 \to x_2 \to x_3 \to x_4$)



(b) Masked Language Modeling (order: $x_0, [\text{MASK}], x_2, [\text{MASK}], x_4 \to x_1, x_3$)



(c) Permuted Language Modeling (order: $x_0 \to x_4 \to x_2 \to x_1 \to x_3$)

Hình 7.3: So sánh kết quả đặt mặt nạ tự chú ý của mô hình hóa ngôn ngữ nhân quả, mô hình hóa ngôn ngữ có che và mô hình hóa ngôn ngữ hoán vị. Ô màu xám biểu thị token ở vị trí j không chú ý tới token ở vị trí i. Ô màu xanh (i, j) biểu thị token ở vị trí j chú ý tới token ở vị trí i. $\mathbf{e}_{\text{mask}}$ biểu diễn vectơ nhúng của ký hiệu [MASK], là tổ hợp của nhúng token và nhúng vị trí.

Chú giải của người dịch: Đọc bảng mặt nạ trong Hình 7.3

Mỗi hàng ứng với một token đang được dự đoán, mỗi cột là một token mà nó có thể "nhìn" tới. Ô xanh = được nhìn, ô xám = bị che mắt. Ở (a), hàng x_3 chỉ xanh ở các cột x_0, x_1, x_2, x_3: nó chỉ thấy quá khứ (tam giác dưới). Ở (c), hàng x_1 xanh ở cột x_0, x_2, x_4 (cùng chính nó) nhưng xám ở x_3 — đúng như $\Pr(x_1|\mathbf{e}_0, \mathbf{e}_4, \mathbf{e}_2)$, vì theo thứ tự $x_0 \to x_4 \to x_2 \to x_1 \to x_3$ thì x_3 chưa được "đoán" khi đến lượt x_1. Như vậy chỉ cần thay đổi mặt nạ, không cần xáo trộn câu.

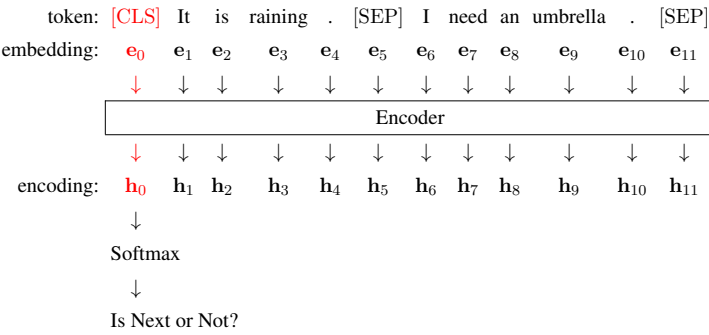
3. Tiền huấn luyện bộ mã hóa như các bộ phân loại

Một cách tiếp cận phổ biến khác để huấn luyện bộ mã hóa là dùng các nhiệm vụ phân loại. Trong học tự giám sát, điều này thường được thực hiện bằng cách tạo ra những thử thách phân loại mới từ văn bản chưa gán nhãn. Có nhiều cách khác nhau để thiết kế các nhiệm vụ phân loại. Ở đây chúng tôi trình bày hai nhiệm vụ phổ biến.

Một phương pháp đơn giản, gọi là **dự đoán câu tiếp theo** (*next sentence prediction*) (**NSP**), được trình bày trong bài báo gốc của BERT [Devlin và cộng sự, 2019]. NSP dựa trên giả định rằng một bộ mã hóa văn bản tốt cần nắm bắt được mối quan hệ giữa hai câu. Để mô hình hóa mối quan hệ này, NSP dùng đầu ra của bộ mã hóa cho hai câu liên tiếp Sent_A và Sent_B để xác định xem Sent_B có phải là câu tiếp theo ngay sau Sent_A hay không. Ví dụ, giả sử $\text{Sent}_A =$ '*It is raining* .' (Trời đang mưa.) và $\text{Sent}_B =$ '*I need an umbrella* .' (Tôi cần một cái ô.). Chuỗi đầu vào của bộ mã hóa có thể là

[CLS] It is raining . [SEP] I need an umbrella . [SEP]

trong đó [CLS] là ký hiệu bắt đầu (tức x_0) thường được dùng trong tiền huấn luyện bộ mã hóa, và [SEP] là một ký hiệu phân tách (*separator*) ngăn cách hai câu. Việc xử lý chuỗi này tuân theo quy trình mã hóa Transformer tiêu chuẩn: đầu tiên ta biểu diễn mỗi token x_i bằng vectơ nhúng tương ứng $\mathbf{e}_i$, rồi đưa chuỗi vectơ nhúng $\{\mathbf{e}_0, \cdots, \mathbf{e}_m\}$ vào bộ mã hóa để thu được chuỗi đầu ra $\{\mathbf{h}_0, \cdots, \mathbf{h}_m\}$. Vì $\mathbf{h}_0$ thường được xem là biểu diễn của toàn bộ chuỗi, ta thêm một tầng Softmax lên trên nó để xây dựng một hệ thống phân loại nhị phân (*binary classification*). Quá trình này được minh họa như sau:



(token / embedding / encoding = token / vectơ nhúng / mã hóa; Encoder = bộ mã hóa; Is Next or Not? = có phải câu tiếp theo hay không?)

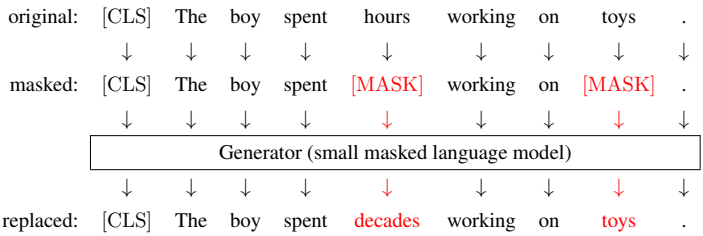
Để tạo các mẫu huấn luyện, ta trích các cặp câu từ một kho ngữ liệu văn bản. Một mẫu dương (*positive sample*) được tạo bằng cách chọn hai câu thực sự liên tiếp nhau, còn một mẫu âm (*negative sample*) được tạo bằng cách ghép một câu với một câu được chọn ngẫu nhiên từ kho ngữ liệu. Nhờ vậy, việc huấn luyện mô hình này giống hệt như huấn luyện một bộ phân loại. Thông thường, NSP được dùng như một hàm mất mát huấn luyện bổ sung cho tiền huấn luyện dựa trên mô hình hóa ngôn ngữ có che.

Chú giải của người dịch: Ví dụ mẫu dương và mẫu âm cho NSP

Mẫu dương: "Trời đang mưa." + "Tôi cần một cái ô." → nhãn *Đúng là câu tiếp theo*. Mẫu âm: "Trời đang mưa." + "Giá vàng hôm nay giảm nhẹ." (câu lấy ngẫu nhiên ở chỗ khác) → nhãn *Không phải*. Nhãn được tạo tự động từ chính văn bản, không cần ai gán — đó là lý do đây vẫn là học *tự giám sát*. "Nhị phân" nghĩa là chỉ có hai nhãn.

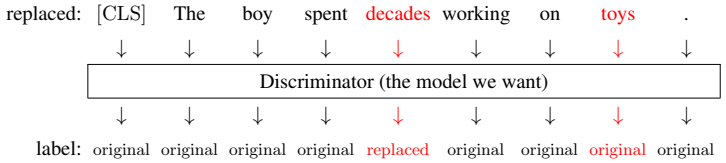
Cách tiếp cận thứ hai để huấn luyện bộ mã hóa Transformer như bộ phân loại là áp dụng các tín hiệu giám sát dựa trên phân loại ở mức token. Ví dụ, Clark và cộng sự [2019b] đề xuất, trong mô hình ELECTRA của họ, huấn luyện một bộ mã hóa Transformer để nhận biết mỗi token đầu vào có giống token gốc hay đã bị thay đổi theo cách nào đó. Bước đầu tiên của phương pháp này là sinh ra một chuỗi mới từ một chuỗi token cho trước, trong đó một số token bị thay đổi. Để làm điều này, ta dùng một mô hình ngôn ngữ có che cỡ nhỏ (gọi là bộ sinh (*generator*)): ta che ngẫu nhiên một số token và huấn luyện mô hình này dự đoán các token bị che. Với mỗi mẫu huấn luyện, mô hình ngôn ngữ có che này đưa ra

một token tại mỗi vị trí bị che, token này có thể khác token gốc. Đồng thời, ta huấn luyện một bộ mã hóa Transformer khác (gọi là bộ phân biệt (*discriminator*)) để xác định mỗi token được dự đoán giống token gốc hay đã bị thay đổi. Cụ thể hơn, ta dùng bộ sinh để sinh ra một chuỗi trong đó một số token bị thay thế. Dưới đây là một minh họa.



(original = gốc; masked = bị che; replaced = sau thay thế; Generator (small masked language model) = bộ sinh (mô hình ngôn ngữ có che cỡ nhỏ). Câu gốc: "Cậu bé đã dành hàng giờ làm đồ chơi."; sau thay thế: "Cậu bé đã dành hàng chục năm làm đồ chơi.")

Sau đó, ta dùng bộ phân biệt để gán nhãn cho mỗi token là original (gốc) hay replaced (bị thay thế), như sau



(Discriminator (the model we want) = bộ phân biệt (mô hình ta cần); label = nhãn)

Chú giải của người dịch: Vì sao "toys" vẫn có nhãn original?

Ở vị trí bị che thứ hai, bộ sinh đoán ra đúng từ gốc *toys*, nên dù vị trí này từng bị che, token cuối cùng vẫn trùng với bản gốc và được gán nhãn *original*. Chỉ *decades* (thay cho *hours*) mới là *replaced*. Bộ phân biệt giống một "thám tử" phải tìm ra những từ bị đánh tráo trong câu; để làm tốt, nó phải hiểu ngữ cảnh rất kỹ — và chính khả năng hiểu này là thứ ta muốn có ở một bộ mã hóa tốt. Khác với BERT chỉ học từ khoảng 15% token bị che, ELECTRA nhận tín hiệu học ở *mọi* token trong câu.

Khi huấn luyện, bộ sinh được tối ưu hóa như một mô hình ngôn ngữ có che bằng ước lượng hợp lý cực đại, còn bộ phân biệt được tối ưu hóa như một bộ phân loại bằng hàm mất mát dựa trên phân loại. Trong ELECTRA, hàm mất mát dựa trên hợp lý cực đại và hàm mất mát dựa trên phân loại được kết hợp để huấn luyện đồng thời cả bộ sinh lẫn bộ phân biệt. Một cách tiếp cận khác là dùng mạng đối sinh (*generative adversarial networks*) (GAN), tức là bộ sinh được huấn luyện để đánh lừa bộ phân biệt, còn bộ phân biệt được huấn luyện để phân biệt đầu ra của bộ sinh với phân phối thật. Tuy nhiên, kiểu huấn luyện GAN làm nhiệm vụ huấn luyện phức tạp hơn và khó mở rộng quy mô hơn. Dù sao, một khi việc huấn luyện hoàn tất, bộ sinh sẽ bị loại bỏ, và phần mã hóa của bộ phân biệt được dùng làm mô hình đã tiền huấn luyện cho các nhiệm vụ hạ nguồn.

7.2.3 Tiền huấn luyện bộ mã hóa–bộ giải mã

Trong NLP, kiến trúc bộ mã hóa–bộ giải mã thường được dùng để mô hình hóa các bài toán chuỗi-sang-chuỗi (*sequence-to-sequence*), chẳng hạn dịch máy và hỏi đáp. Ngoài những bài toán chuỗi-sang-chuỗi điển hình này trong NLP, các mô hình bộ mã hóa–bộ giải mã có thể được mở rộng để xử lý nhiều bài toán khác. Một ý tưởng đơn giản là coi văn bản vừa là đầu vào vừa là đầu ra của một bài toán, nhờ đó ta có thể áp dụng trực tiếp các mô hình bộ mã hóa–bộ giải mã. Ví dụ, cho một văn bản đầu vào, ta có thể chỉ dẫn mô hình sinh ra văn bản mô tả cảm xúc của nó (chẳng hạn *positive* – tích cực, *negative* – tiêu cực, hoặc *neutral* – trung tính).

Ý tưởng như vậy cho phép ta phát triển một hệ thống văn bản-sang-văn bản (*text-to-text*) duy nhất để giải quyết bất kỳ bài toán NLP nào. Ta có thể đưa các bài toán khác nhau về cùng một định dạng văn bản-sang-văn bản. Đầu tiên, ta huấn luyện một mô hình bộ mã hóa–bộ giải mã để thu nhận tri thức đa dụng về ngôn ngữ thông qua tự giám sát. Mô hình này sau đó được tinh chỉnh cho các nhiệm vụ hạ nguồn cụ thể bằng dữ liệu văn bản-sang-văn bản tương ứng.

1. Tiền huấn luyện bộ mã hóa–bộ giải mã có che

Trong mô hình **T5** của Raffel và cộng sự [2020], nhiều nhiệm vụ khác nhau được đưa về cùng một nhiệm vụ văn bản-sang-văn bản. Mỗi mẫu trong T5 tuân theo định dạng

$$\text{Văn bản nguồn} \quad \rightarrow \quad \text{Văn bản đích}$$

Ở đây $\rightarrow$ ngăn cách văn bản nguồn, gồm một mô tả nhiệm vụ hay chỉ dẫn cùng với đầu vào đưa cho hệ thống, với văn bản đích, là câu trả lời cho nhiệm vụ đầu vào. Ví dụ, xét nhiệm vụ dịch từ tiếng Trung sang tiếng Anh. Một mẫu huấn luyện có thể được biểu diễn như sau:

[CLS] Dịch từ tiếng Trung sang tiếng Anh: 你好! $\rightarrow$ $\langle s \rangle$ Hello!

trong đó [CLS] và $\langle s \rangle$ lần lượt là các ký hiệu bắt đầu ở phía nguồn và phía đích[5].

Tương tự, ta có thể biểu diễn các nhiệm vụ khác theo cùng cách này. Ví dụ

[CLS] Trả lời: Albert Einstein sinh năm nào?
$\rightarrow$ $\langle s \rangle$ Ông sinh ngày 14 tháng 3 năm 1879.

[CLS] Rút gọn câu: vị giáo sư, người đã công bố rất nhiều bài báo trong lĩnh vực của mình, sẽ có một bài giảng về chủ đề này vào tuần tới.
$\rightarrow$ $\langle s \rangle$ Vị giáo sư giàu kinh nghiệm sẽ giảng bài vào tuần tới.

[CLS] Văn bản: John đã mua một chiếc xe mới. Giả thuyết: John có một chiếc xe.
$\rightarrow$ $\langle s \rangle$ Kéo theo (*Entailment*)

[CLS] Chấm điểm bản dịch từ tiếng Anh sang tiếng Trung. Tiếng Anh: when in Rome, do as the Romans do. Tiếng Trung: 人 在 罗马 就 像 罗马 人 一样 做事 。
$\rightarrow$ $\langle s \rangle$ 0.81

trong đó các chỉ dẫn được tô xám. Điều thú vị là trong ví dụ cuối cùng, ta đưa một bài toán chấm điểm liên tục về dạng một nhiệm vụ sinh văn

[5]Ta có thể dùng cùng một ký hiệu bắt đầu cho các chuỗi khác nhau. Ở đây chúng tôi dùng các ký hiệu khác nhau để phân biệt các chuỗi ở phía bộ mã hóa và phía bộ giải mã.

bản. Mục tiêu của ta là sinh ra chuỗi văn bản biểu diễn con số 0.81, thay vì xuất ra một giá trị số liên tục.

Chú giải của người dịch: Giải nghĩa các ví dụ

Ví dụ thứ ba là nhiệm vụ *suy diễn ngôn ngữ tự nhiên*: cho một văn bản và một giả thuyết, mô hình phải cho biết văn bản có "kéo theo" (*entailment*) giả thuyết hay không — ở đây có, vì đã mua xe thì có xe. Ví dụ thứ tư chấm chất lượng bản dịch tục ngữ "when in Rome, do as the Romans do" (nhập gia tùy tục) trên thang từ 0 đến 1. Điểm mấu chốt: mọi nhiệm vụ, dù là trả lời câu hỏi, rút gọn câu, phân loại hay chấm điểm, đều được viết thành "văn bản vào → văn bản ra", nên chỉ cần *một* mô hình cho tất cả.

Cách tiếp cận mô tả ở trên mang lại một khuôn khổ mới cho việc hiểu và sinh ngôn ngữ phổ quát. Cả chỉ dẫn nhiệm vụ lẫn đầu vào của bài toán đều được cung cấp cho hệ thống dưới dạng văn bản. Hệ thống sau đó làm theo chỉ dẫn để hoàn thành nhiệm vụ. Phương pháp này gộp các bài toán khác nhau lại với nhau, với lợi ích là huấn luyện một mô hình duy nhất có thể thực hiện nhiều nhiệm vụ cùng lúc.

Nhìn chung, việc tinh chỉnh là cần thiết để thích nghi mô hình đã tiền huấn luyện cho một nhiệm vụ hạ nguồn cụ thể. Trong quá trình này, có thể dùng những cách khác nhau để chỉ dẫn mô hình về nhiệm vụ, chẳng hạn dùng tên ngắn của nhiệm vụ làm tiền tố cho chuỗi đầu vào thực tế, hoặc cung cấp một mô tả chi tiết về nhiệm vụ. Vì chỉ dẫn nhiệm vụ được cung cấp dưới dạng văn bản và được xử lý như một phần của đầu vào, năng lực làm theo chỉ dẫn tổng quát có thể được thu nhận ngay trong giai đoạn tiền huấn luyện. Điều này hỗ trợ rất nhiều cho học zero-shot. Chẳng hạn, các mô hình đã tiền huấn luyện thường có thể tổng quát hóa để giải những bài toán mới mà chỉ dẫn riêng cho nhiệm vụ đó chưa từng xuất hiện trong quá trình huấn luyện.

Đã có một số phương pháp học tự giám sát mạnh mẽ cho bộ mã hóa hoặc bộ giải mã Transformer. Áp dụng các phương pháp này để tiền huấn luyện các mô hình bộ mã hóa–bộ giải mã là tương đối đơn giản. Một lựa chọn phổ biến là huấn luyện các mô hình bộ mã hóa–bộ giải mã như các mô hình ngôn ngữ. Ví dụ, bộ mã hóa nhận một tiền tố (*prefix*) của chuỗi, còn bộ giải mã sinh ra phần còn lại của chuỗi. Tuy nhiên, điều này khác với mô hình hóa ngôn ngữ nhân quả tiêu chuẩn, trong đó toàn bộ chuỗi được sinh theo kiểu tự hồi quy bắt đầu từ token đầu tiên. Trong trường hợp

của ta, bộ mã hóa xử lý tiền tố cùng một lúc, sau đó bộ giải mã dự đoán các token tiếp theo theo cách của mô hình hóa ngôn ngữ nhân quả. Nói chính xác hơn, đây là một bài toán **mô hình hóa ngôn ngữ tiền tố** (*prefix language modeling*): mô hình ngôn ngữ dự đoán phần chuỗi tiếp theo khi biết một tiền tố, đóng vai trò ngữ cảnh cho việc dự đoán.

Xét ví dụ sau

$$\underbrace{\text{[CLS] The puppies are frolicking}}_{\text{Tiền tố}} \rightarrow \underbrace{\langle s \rangle \text{ outside the house .}}_{\text{Phần chuỗi tiếp theo}}$$

(câu đầy đủ "The puppies are frolicking outside the house ." nghĩa là "Những chú cún con đang nô đùa bên ngoài ngôi nhà."). Ta có thể huấn luyện trực tiếp một mô hình bộ mã hóa–bộ giải mã bằng những ví dụ như thế này. Khi đó, bộ mã hóa học cách hiểu tiền tố, còn bộ giải mã học cách viết tiếp dựa trên sự hiểu đó. Đối với tiền huấn luyện quy mô lớn, rất dễ tạo ra một số lượng lớn ví dụ huấn luyện từ văn bản chưa gán nhãn.

Cần lưu ý rằng để các mô hình bộ mã hóa–bộ giải mã đã tiền huấn luyện hoạt động hiệu quả trong các nhiệm vụ đa ngôn ngữ (*multilingual*) và xuyên ngôn ngữ (*cross-lingual*) (ví dụ dịch máy), chúng cần được huấn luyện trên dữ liệu đa ngôn ngữ. Điều này thường đòi hỏi bộ từ vựng phải chứa các token của nhiều ngôn ngữ. Bằng cách đó, mô hình có thể học được các biểu diễn dùng chung giữa các ngôn ngữ khác nhau, từ đó có năng lực cả hiểu lẫn sinh ngôn ngữ trong bối cảnh đa ngôn ngữ và xuyên ngôn ngữ.

Cách tiếp cận thứ hai để tiền huấn luyện các mô hình bộ mã hóa–bộ giải mã là mô hình hóa ngôn ngữ có che. Trong cách tiếp cận này, như đã bàn ở Mục 7.2.2, các token trong một chuỗi được thay ngẫu nhiên bằng một ký hiệu che, rồi mô hình được huấn luyện để dự đoán các token bị che này dựa trên chuỗi bị làm hỏng.

Để minh họa, xét nhiệm vụ che và tái tạo câu

The puppies are frolicking outside the house .

Bằng cách che hai token (chẳng hạn *frolicking* và *the*), ta có đầu vào và đầu ra của mô hình theo kiểu BERT như sau

[CLS] The puppies are [MASK] outside [MASK] house .
$\rightarrow$ $\langle s \rangle$ ___ ___ ___ frolicking ___ the ___ ___

Ở đây __ biểu thị vị trí bị che mà tại đó ta không đưa ra dự đoán token. Bằng cách thay đổi tỉ lệ phần trăm token bị che trong văn bản, cách tiếp cận này có thể được tổng quát hóa về phía huấn luyện kiểu BERT hoặc huấn luyện kiểu mô hình hóa ngôn ngữ [Song và cộng sự, 2019]. Ví dụ, nếu ta che tất cả các token, thì mô hình được huấn luyện để sinh ra toàn bộ chuỗi

[CLS] [MASK] [MASK] [MASK] [MASK] [MASK] [MASK] [MASK] [MASK]

→ ⟨s⟩ The puppies are frolicking outside the house .

Trong trường hợp này, ta huấn luyện bộ giải mã như một mô hình ngôn ngữ.

Lưu ý rằng, trong bối cảnh kiến trúc bộ mã hóa–bộ giải mã, ta có thể dùng bộ mã hóa để đọc chuỗi bị che, và dùng bộ giải mã để dự đoán chuỗi gốc. Với mục tiêu này, về bản chất ta có một bộ tự mã hóa khử nhiễu (*denoising autoencoder*): bộ mã hóa biến đổi đầu vào bị làm hỏng thành một biểu diễn ẩn nào đó, và bộ giải mã tái tạo đầu vào chưa bị làm hỏng từ biểu diễn ẩn này. Đây là một ví dụ về đầu vào và đầu ra cho huấn luyện khử nhiễu.

[CLS] The puppies are [MASK] outside [MASK] house .

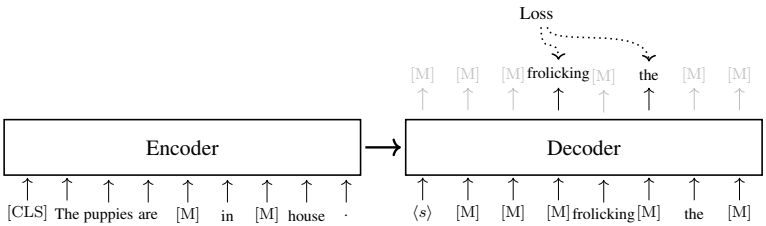
→ ⟨s⟩ The puppies are frolicking outside the house .

Bằng cách học ánh xạ từ chuỗi bị làm hỏng này sang bản gốc chưa bị làm hỏng của nó, mô hình có được khả năng hiểu ở phía bộ mã hóa và khả năng sinh ở phía bộ giải mã. Xem Hình 7.4 để thấy minh họa cách một mô hình bộ mã hóa–bộ giải mã được huấn luyện với mục tiêu kiểu BERT và mục tiêu tự mã hóa khử nhiễu.

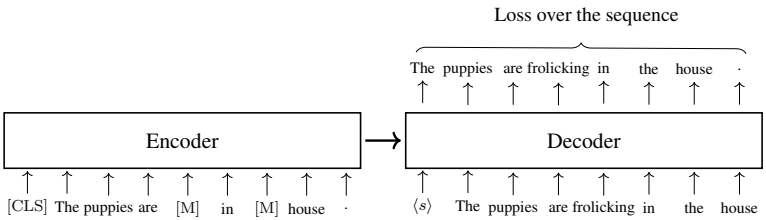
Giải nghĩa nhãn trong hình: Encoder/Decoder = bộ mã hóa/bộ giải mã; Loss = mất mát; Loss over the sequence = mất mát trên toàn chuỗi; (a) huấn luyện mô hình bộ mã hóa–bộ giải mã với mô hình hóa ngôn ngữ có che kiểu BERT; (b) huấn luyện với tự mã hóa khử nhiễu.

Chú giải của người dịch: Khử nhiễu là gì?

"Nhiễu" ở đây là những hư hỏng ta cố ý gây ra cho câu (che từ, xóa từ, đảo thứ tự…). Bộ tự mã hóa khử nhiễu giống như người phục chế một bức ảnh cũ bị rách: nhìn bản hỏng, phải vẽ lại bản nguyên vẹn. Muốn phục chế đúng,



(a) Training an encoder-decoder model with BERT-style masked language modeling



(b) Training an encoder-decoder model with denoising autoencoding

Hình 7.4: Minh họa việc huấn luyện một mô hình bộ mã hóa–bộ giải mã bằng phương pháp kiểu BERT và phương pháp tự mã hóa khử nhiễu. Trong cả hai phương pháp, đầu vào của bộ mã hóa là một chuỗi token bị làm hỏng, trong đó một số token bị che và thay bằng [MASK] (viết tắt là [M]). Bộ giải mã dự đoán các token bị che này, nhưng theo những cách khác nhau. Trong huấn luyện kiểu BERT, bộ giải mã chỉ cần tính mất mát cho các token bị che, còn các token còn lại trong chuỗi có thể đơn giản được coi là token [MASK]. Trong tự mã hóa khử nhiễu, bộ giải mã dự đoán chuỗi gồm tất cả các token theo kiểu tự hồi quy. Kết quả là mất mát được tính bằng cách cộng dồn mất mát của tất cả các token này, như trong mô hình hóa ngôn ngữ tiêu chuẩn.

người đó phải hiểu bức ảnh vẽ gì. Tương tự, để tái tạo câu gốc, mô hình buộc phải học ngữ pháp và ý nghĩa. Khác biệt giữa (a) và (b) trong Hình 7.4: ở (a) chỉ chấm điểm tại 2 ô bị che; ở (b) bộ giải mã viết lại cả câu, và mọi token đều góp vào mất mát.

Khi chọn ngẫu nhiên các token để che, ta có thể che các token liên tiếp nhau [Joshi và cộng sự, 2020]. Đây là một ví dụ.

[CLS] The puppies are [MASK] outside [MASK] [MASK] .
→ ⟨s⟩ The puppies are frolicking outside the house .

Một cách khác để xử lý các token bị che liên tiếp là biểu diễn chúng dưới dạng các đoạn (*span*). Ở đây ta theo công trình của Raffel và cộng sự [2020], dùng [X], [Y] và [Z] để ký hiệu các token canh gác (*sentinel token*) bao trùm một hoặc nhiều token bị che liên tiếp. Với ký hiệu này, ta có thể viết lại ví dụ huấn luyện ở trên thành:

[CLS] The puppies are [X] outside [Y] .
→ ⟨s⟩ [X] frolicking [Y] the house [Z]

Ý tưởng là ta biểu diễn chuỗi bị làm hỏng như một chuỗi chứa các ô giữ chỗ. Nhiệm vụ huấn luyện là điền vào các ô này những token đúng bằng cách dùng ngữ cảnh xung quanh. Một ưu điểm của cách tiếp cận này là các chuỗi dùng trong huấn luyện sẽ ngắn hơn, giúp việc huấn luyện hiệu quả hơn. Lưu ý rằng mô hình hóa ngôn ngữ có che cung cấp một khuôn khổ rất tổng quát để huấn luyện các mô hình bộ mã hóa–bộ giải mã. Có thể điều chỉnh nhiều thiết lập khác nhau để có những phiên bản huấn luyện khác nhau, chẳng hạn thay đổi tỉ lệ token bị che và độ dài tối đa của các đoạn bị che.

Chú giải của người dịch: Đọc ví dụ với token canh gác

Ở đầu vào, [X] thay cho "frolicking" và [Y] thay cho "the house". Đầu ra chỉ cần liệt kê: "[X] là frolicking, [Y] là the house", và [Z] đánh dấu kết thúc. Bộ giải mã không phải chép lại các từ không bị che (The, puppies, are, outside), nên chuỗi đích ngắn hơn hẳn — đó là lý do huấn luyện nhanh hơn.

2. Huấn luyện khử nhiễu

Nếu ta xem tiền huấn luyện bộ mã hóa–bộ giải mã là một bài toán tự mã hóa khử nhiễu, thì có rất nhiều cách làm hỏng đầu vào. Chẳng hạn, ngoài việc che token ngẫu nhiên, ta còn có thể sửa đổi token hoặc sắp xếp lại thứ tự của chúng.

Giả sử ta có một mô hình bộ mã hóa–bộ giải mã có thể ánh xạ một chuỗi đầu vào $\mathbf{x}$ thành một chuỗi đầu ra $\mathbf{y}$

$$\begin{aligned}\mathbf{y} &= \text{Decode}_\omega(\text{Encode}_\theta(\mathbf{x})) \\ &= \text{Model}_{\theta,\omega}(\mathbf{x}),\end{aligned} \tag{7.15}$$

trong đó θ và ω lần lượt là tham số của bộ mã hóa và bộ giải mã. Trong tự mã hóa khử nhiễu, ta thêm một ít nhiễu vào $\mathbf{x}$ để thu được một đầu vào bị nhiễu, bị làm hỏng $\mathbf{x}_{\text{noise}}$. Bằng cách đưa $\mathbf{x}_{\text{noise}}$ vào bộ mã hóa, bộ giải mã được huấn luyện để tái tạo đầu vào gốc. Mục tiêu huấn luyện có thể được định nghĩa như sau:

$$(\hat{\theta}, \hat{\omega}) = \arg\min_{\theta,\omega} \text{Loss}(\text{Model}_{\theta,\omega}(\mathbf{x}_{\text{noise}}), \mathbf{x}), \tag{7.16}$$

Ở đây hàm mất mát $\text{Loss}(\text{Model}_{\theta,\omega}(\mathbf{x}_{\text{noise}}), \mathbf{x})$ đánh giá mức độ mô hình $\text{Model}_{\theta,\omega}(\mathbf{x}_{\text{noise}})$ tái tạo tốt đầu vào gốc $\mathbf{x}$ đến đâu. Như thường lệ, ta có thể chọn hàm mất mát entropy chéo.

Với kiến trúc mô hình và khuôn khổ huấn luyện đã có, vấn đề còn lại là làm hỏng đầu vào như thế nào. Lewis và cộng sự [2020a], trong mô hình **BART** của họ, đề xuất một số cách làm hỏng chuỗi đầu vào.

- **Che token** (*Token Masking*). Đây chính là phương pháp che mà ta đã dùng trong mô hình hóa ngôn ngữ có che. Các token được chọn ngẫu nhiên và thay bằng [MASK].
- **Xóa token** (*Token Deletion*). Phương pháp này tương tự che token. Tuy nhiên, thay vì thay các token được chọn bằng ký hiệu đặc biệt [MASK], ta xóa hẳn các token này khỏi chuỗi. Xem ví dụ sau để so sánh phương pháp che token và xóa token.

Gốc ($\mathbf{x}$):	The puppies are frolicking outside the house .
Che token ($\mathbf{x}_{\text{noise}}$):	The puppies are [MASK] outside [MASK] house .
Xóa token ($\mathbf{x}_{\text{noise}}$):	The puppies are ~~frolicking~~ outside ~~the~~ house .

trong đó các token được gạch dưới trong chuỗi gốc là các token bị che hoặc bị xóa.

- **Che đoạn** (*Span Masking*). Các đoạn không chồng lấn nhau được lấy mẫu ngẫu nhiên trên chuỗi. Mỗi đoạn được che bằng [MASK]. Ta cũng cho phép các đoạn có độ dài 0, và trong những trường hợp đó, [MASK] đơn giản được chèn vào một vị trí trong chuỗi. Ví dụ, ta có thể dùng che đoạn để làm hỏng chuỗi trên như sau

  Gốc ($\mathbf{x}$): The $\underline{0}$ puppies are $\underline{\text{frolicking outside the}}$ house .
  Che đoạn ($\mathbf{x}_{\text{noise}}$): The [MASK] puppies are [MASK] house .

  Ở đây đoạn *frolicking outside the* được thay bằng một [MASK] duy nhất. $\underline{0}$ biểu thị một đoạn độ dài 0, vì vậy ta chèn một [MASK] vào giữa *The* và *puppies*. Che đoạn đặt ra những thử thách dự đoán mới, trong đó mô hình cần biết có bao nhiêu token được sinh ra từ một đoạn. Bài toán này rất giống với mô hình hóa độ sinh sản (*fertility modeling*) trong dịch máy [Brown và cộng sự, 1993].

Nếu ta xét một chuỗi gồm nhiều câu, có thể áp dụng thêm các phương pháp làm hỏng khác. Trong mô hình BART có hai phương pháp như vậy.

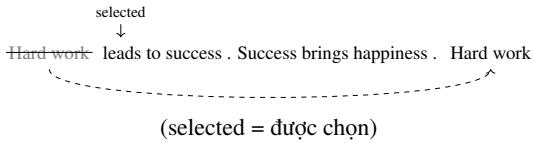
- **Sắp xếp lại câu** (*Sentence Reordering*). Phương pháp này hoán vị ngẫu nhiên các câu để mô hình có thể học cách sắp xếp lại các câu trong một văn bản. Ví dụ, xét hai câu liên tiếp

  Hard work leads to success . Success brings happiness .
  (Chăm chỉ dẫn tới thành công. Thành công mang lại hạnh phúc.)

  Ta có thể đảo thứ tự hai câu để có một chuỗi đầu vào bị làm hỏng

  Success brings happiness . Hard work leads to success .

- **Xoay văn bản** (*Document Rotation*). Mục tiêu của nhiệm vụ này là xác định token bắt đầu của chuỗi. Đầu tiên, một token được chọn ngẫu nhiên từ chuỗi. Sau đó, chuỗi được xoay vòng sao cho token được chọn trở thành token đầu tiên. Ví dụ, giả sử ta chọn token *leads* từ chuỗi ở trên. Chuỗi sau khi xoay là



trong đó dãy con *Hard work* đứng trước *leads* được nối vào cuối chuỗi.

Khi tiền huấn luyện, ta có thể áp dụng nhiều phương pháp làm hỏng để học được những mô hình bền vững (*robust*), chẳng hạn chọn ngẫu nhiên một trong số chúng cho mỗi mẫu huấn luyện. Trong thực tế, kết quả của tiền huấn luyện bộ mã hóa–bộ giải mã phụ thuộc rất nhiều vào các phương pháp làm hỏng đầu vào được sử dụng, vì vậy ta thường cần chọn các mục tiêu huấn luyện thích hợp thông qua thực nghiệm cẩn thận.

Chú giải của người dịch: Vì sao làm hỏng câu theo nhiều cách?

Mỗi kiểu làm hỏng "ép" mô hình học một kỹ năng khác: che/xóa token buộc mô hình đoán từ còn thiếu (với xóa token, mô hình còn phải tự tìm ra *chỗ* bị xóa); che đoạn buộc nó đoán cả *số lượng* từ bị mất; sắp xếp lại câu buộc nó hiểu mạch lạc giữa các câu (câu nào là nguyên nhân, câu nào là kết quả); xoay văn bản buộc nó nhận ra đâu là điểm bắt đầu tự nhiên của một đoạn văn. Giống như ôn thi bằng nhiều dạng bài tập khác nhau thì hiểu bài sâu hơn.

7.2.4 So sánh các nhiệm vụ tiền huấn luyện

Đến đây, chúng ta đã bàn về nhiều nhiệm vụ tiền huấn luyện khác nhau. Vì cùng một mục tiêu huấn luyện thường có thể được áp dụng cho các kiến trúc khác nhau (ví dụ, dùng mô hình hóa ngôn ngữ có che cho cả tiền huấn luyện chỉ với bộ mã hóa lẫn tiền huấn luyện bộ mã hóa–bộ giải mã), việc phân loại các nhiệm vụ này chỉ dựa trên kiến trúc mô hình là chưa đủ. Thay vào đó, ta tóm tắt chúng dựa trên mục tiêu huấn luyện.

- **Mô hình hóa ngôn ngữ** (*Language Modeling*). Thông thường, cách tiếp cận này chỉ một quá trình sinh chuỗi tự hồi quy. Tại mỗi bước, mô hình dự đoán token tiếp theo chỉ dựa trên ngữ cảnh đứng trước nó.
- **Mô hình hóa ngôn ngữ có che** (*Masked Language Modeling*). Mô

hình hóa ngôn ngữ có che tuân theo một khuôn khổ tổng quát "che rồi dự đoán". Nó che ngẫu nhiên các token trong một chuỗi và dự đoán các token này bằng cách dùng toàn bộ chuỗi đã bị che.

- **Mô hình hóa ngôn ngữ hoán vị** (*Permuted Language Modeling*). Mô hình hóa ngôn ngữ hoán vị có nhiều điểm giống mô hình hóa ngôn ngữ có che, nhưng loại bỏ ràng buộc cố định từ trái sang phải bằng cách hoán vị thứ tự phân tích thành tích. Nó giữ nguyên chuỗi đầu vào nhưng sắp xếp lại thứ tự phân tích (dự đoán). Mỗi dự đoán dựa trên một số token ngữ cảnh được chọn ngẫu nhiên.

- **Huấn luyện phân biệt** (*Discriminative Training*). Trong huấn luyện phân biệt, tín hiệu giám sát được tạo ra từ các nhiệm vụ phân loại. Các mô hình đã tiền huấn luyện được tích hợp vào các bộ phân loại và được huấn luyện cùng với các phần còn lại của bộ phân loại để nâng cao hiệu năng phân loại.

- **Tự mã hóa khử nhiễu** (*Denoising Autoencoding*). Cách tiếp cận này được áp dụng cho tiền huấn luyện các mô hình bộ mã hóa–bộ giải mã. Đầu vào là một chuỗi bị làm hỏng (với các loại nhiễu như che, xóa hoặc hoán vị), và mô hình được huấn luyện để tái tạo chuỗi gốc.

Bảng 7.1 minh họa các phương pháp này và các biến thể của chúng bằng ví dụ. Mặc dù định dạng đầu vào/đầu ra trong các ví dụ nhìn chung không phụ thuộc vào mô hình, chúng tôi chỉ rõ các kiến trúc mô hình tương thích với mỗi nhiệm vụ tiền huấn luyện. Trong mỗi ví dụ, đầu vào là một chuỗi token, còn đầu ra là một chuỗi token hoặc một số xác suất. Với các nhiệm vụ sinh, chẳng hạn mô hình hóa ngôn ngữ, các chỉ số trên được dùng để chỉ thứ tự sinh ở phía đích. Nếu các chỉ số trên bị lược bỏ, điều đó có nghĩa là các token đầu ra có thể được dự đoán hoặc theo kiểu tự hồi quy hoặc song song (không tự hồi quy). Ở phía nguồn, ta giả định rằng chuỗi trải qua quá trình mã hóa Transformer tiêu chuẩn, nghĩa là mỗi token có thể nhìn thấy toàn bộ chuỗi trong cơ chế tự chú ý. Ngoại lệ duy nhất là mô hình hóa ngôn ngữ hoán vị, trong đó quá trình sinh tự hồi quy được thực hiện bằng cách đặt các mặt nạ chú ý ở phía bộ mã hóa. Để đơn giản hóa thảo luận, ta bỏ token $\langle s \rangle$ khỏi phía đích của mỗi ví dụ.

Phương pháp	Enc	Dec	E-D	Đầu vào	Đầu ra
LM nhân quả		•	•		The^1 kitten^2 is^3 chasing^4 the^5 ball^6 .^{7}
LM tiền tố		•	•	[C] The kitten is	chasing^1 the^2 ball^3 .^{4}
LM có che	•		•	[C] The kitten [M] chasing the [M] .	_ _ is _ _ ball _
Kiểu MASS	•		•	[C] The kitten [M] [M] [M] ball .	_ _ is chasing the _ _
Kiểu BERT	•		•	[C] The kitten [M] playing the [M] .	_ kitten is chasing _ ball _
LM hoán vị	•			[C] The kitten is chasing the ball .	The^5 kitten^7 is^6 chasing^1 the^4 ball^2 .^{3}
Dự đoán câu tiếp theo	•			[C] The kitten is chasing the ball . Birds eat worms .	Pr(IsNext | representation-of-[C])
So sánh câu	•			Sent$_a$: The kitten is chasing the ball . Sent$_b$: A cat is running after a toy .	Score(Sent$_a$, Sent$_b$)
Phân loại token	•			[C] The kitten is chasing the ball .	Pr(·|The) Pr(·|kitten) … Pr(·|.)
Sắp xếp lại token			•	[C] . kitten the chasing The is ball	The^1 kitten^2 is^3 chasing^4 the^5 ball^6 .^{7}
Xóa token			•	[C] The kitten is ~~chasing~~ the ball .	The^1 kitten^2 is^3 chasing^4 the^5 ball^6 .^{7}
Che đoạn			•	[C] The kitten [M] is [M] .	The^1 kitten^2 is^3 chasing^4 the^5 ball^6 .^{7}
Che bằng token canh gác			•	[C] The kitten [X] the [Y]	[X]^1 is^2 chasing^3 [Y]^4 ball^5 .^{6}
Sắp xếp lại câu			•	[C] The ball rolls away swiftly . The kitten is chasing the ball .	The^1 kitten^2 is^3 chasing^4 the^5 ball^6 .^{7} The^8 ball^9 rolls^{10} away^{11} swiftly^{12} .^{13}
Xoay văn bản			•	[C] chasing the ball . The ball rolls away swiftly . The kitten is	The^1 kitten^2 is^3 chasing^4 the^5 ball^6 .^{7} The^8 ball^9 rolls^{10} away^{11} swiftly^{12} .^{13}

Bảng 7.1: So sánh các nhiệm vụ tiền huấn luyện, gồm mô hình hóa ngôn ngữ, mô hình hóa ngôn ngữ có che, mô hình hóa ngôn ngữ hoán vị, huấn luyện phân biệt và tự mã hóa khử nhiễu. [C] = [CLS], [M] = [MASK], [X], [Y] = token canh gác. Enc, Dec và E-D cho biết cách tiếp cận có thể được áp dụng lần lượt cho mô hình chỉ có bộ mã hóa, chỉ có bộ giải mã, và bộ mã hóa–bộ giải mã hay không. Với các nhiệm vụ sinh, chỉ số trên được dùng để biểu diễn thứ tự của các token. (LM = mô hình ngôn ngữ.)

Chú giải của người dịch: Đọc Bảng 7.1

Các ví dụ dùng câu "The kitten is chasing the ball ." (Chú mèo con đang đuổi theo quả bóng.) và "The ball rolls away swiftly ." (Quả bóng lăn đi thật nhanh.); "Birds eat worms ." (Chim ăn sâu.) là câu ngẫu nhiên không liên quan, dùng làm mẫu âm cho dự đoán câu tiếp theo; "A cat is running after a toy ." (Một con mèo đang chạy theo món đồ chơi.) dùng để chấm độ tương đồng với câu thứ nhất. Chỉ số trên cho biết thứ tự sinh: ở dòng LM hoán vị, "chasing" được đoán đầu tiên (1), rồi "ball" (2), "." (3)…. Dấu _ là vị trí không cần dự đoán. Ở dòng kiểu BERT, từ "playing" tô xám là một token đã bị thay bằng từ ngẫu nhiên (thay cho "chasing" gốc), một kỹ thuật sẽ được trình bày ở Mục 7.3.

Mặc dù các nhiệm vụ tiền huấn luyện này khác nhau, ta vẫn có thể so sánh chúng trong cùng một khuôn khổ và thiết lập thực nghiệm [Dong và cộng sự, 2019; Raffel và cộng sự, 2020; Lewis và cộng sự, 2020a]. Lưu ý rằng chúng tôi không thể liệt kê tất cả các nhiệm vụ tiền huấn luyện ở đây vì có rất nhiều nhiệm vụ như vậy. Để tìm hiểu thêm về các nhiệm vụ tiền huấn luyện, bạn đọc quan tâm có thể tham khảo một số bài tổng quan về chủ đề này [Qiu và cộng sự, 2020b; Han và cộng sự, 2021a].

7.3 Ví dụ: BERT

Trong mục này, chúng ta giới thiệu BERT, một trong những mô hình mã hóa chuỗi (*sequence encoding*) được tiền huấn luyện (*pre-trained*) sử dụng rộng rãi nhất trong NLP.

7.3.1 Mô hình chuẩn

Mô hình BERT chuẩn, do Devlin và cộng sự [2019] đề xuất, là một bộ mã hóa (*encoder*) Transformer hai chiều (*bidirectional*) được huấn luyện bằng hai nhiệm vụ tự giám sát (*self-supervised*): mô hình hóa ngôn ngữ có che (*masked language modeling*) và dự đoán câu tiếp theo (*next sentence prediction*). Hàm mất mát huấn luyện tổng thể đơn giản là tổng các hàm mất mát của hai nhiệm vụ này:

$$\text{Loss}_{\text{BERT}} = \text{Loss}_{\text{mlm}} + \text{Loss}_{\text{nsp}}. \tag{7.17}$$

Như thường lệ trong học sâu (*deep learning*), ta tối ưu các tham số của mô hình bằng cách cực tiểu hóa hàm mất mát này. Trong quá trình huấn luyện, ta lấy mẫu các lô nhỏ (*mini-batch*) từ một kho ngữ liệu (*corpus*) văn bản, tính $\text{Loss}_{\text{BERT}}$ trên mỗi lô, rồi cập nhật lặp đi lặp lại các tham số của mô hình bằng hạ gradient (*gradient descent*) hoặc các biến thể của nó. Quá trình này lặp lại cho đến khi thỏa mãn một tiêu chí dừng, chẳng hạn hàm mất mát huấn luyện hội tụ hoặc đạt tới số bước tối đa đã định trước.

Chú giải của người dịch: Vì sao chỉ cần cộng hai hàm mất mát?

Hàm mất mát là một con số đo mức độ "sai" của mô hình: càng nhỏ càng tốt. Ở đây mô hình phải làm tốt cùng lúc hai bài tập (đoán từ bị che và đoán hai câu có nối tiếp nhau không). Cộng hai mức sai lại thành một con số duy nhất giống như cộng điểm phạt của hai môn thi: muốn tổng điểm phạt nhỏ thì phải làm tốt cả hai môn. Mỗi bước huấn luyện, thuật toán hạ gradient điều chỉnh các tham số một chút theo hướng làm tổng này giảm xuống.

1. Các hàm mất mát

Nói chung, mô hình BERT được dùng để biểu diễn một câu đơn hoặc một cặp câu, nhờ vậy có thể xử lý nhiều bài toán hiểu ngôn ngữ (*language understanding*) hạ nguồn (*downstream*) khác nhau. Trong mục này, ta giả sử biểu diễn đầu vào là một chuỗi chứa hai câu Sent_A và Sent_B, có dạng

$$[\text{CLS}]\ \ \text{Sent}_A\ \ [\text{SEP}]\ \ \text{Sent}_B\ \ [\text{SEP}]$$

Ở đây ta theo cách ký hiệu của [Devlin và cộng sự, 2019] và dùng $[\text{SEP}]$ để chỉ ký hiệu phân tách (*separator*).

Với chuỗi này, ta có thể tính riêng Loss_{mlm} và Loss_{nsp}. Với mô hình hóa ngôn ngữ có che, một tập con các token (thường là 15%) được chọn ngẫu nhiên để dự đoán. Sau đó chuỗi được sửa đổi theo ba cách:

- **Che token** (*Token Masking*). 80% số token được chọn bị che và thay bằng ký hiệu $[\text{MASK}]$. Ví dụ

Gốc: [CLS] It is raining . [SEP] I need an umbrella . [SEP]
Đã che: [CLS] It is [MASK] . [SEP] I need [MASK] umbrella . [SEP]

trong đó các token được chọn được gạch dưới. Việc dự đoán các token bị che khuyến khích mô hình học các biểu diễn từ ngữ cảnh xung quanh.

- **Thay thế ngẫu nhiên** (*Random Replacement*). 10% số token được chọn bị đổi thành một token ngẫu nhiên. Ví dụ

Gốc:	[CLS] It is raining . [SEP] I need an umbrella . [SEP]
Token ngẫu nhiên:	[CLS] It is raining . [SEP] I need a hat . [SEP]

Điều này giúp mô hình học cách khôi phục một token từ đầu vào đã bị làm hỏng.

- **Giữ nguyên** (*Unchanged*). 10% số token được chọn được giữ nguyên. Ví dụ,

Gốc:	[CLS] It is raining . [SEP] I need an umbrella . [SEP]
Token giữ nguyên:	[CLS] It is raining . [SEP] I need an umbrella . [SEP]

Mặc dù dự đoán một token không bị thay đổi là việc dễ dàng, chiến lược này lại rất quan trọng. Nó khuyến khích mô hình duy trì một biểu diễn có ý nghĩa cho chính từ thực sự quan sát được, qua đó giúp giảm bớt sự chênh lệch giữa tiền huấn luyện (có dùng [MASK]) và tinh chỉnh (*fine-tuning*) (không dùng).

Chú giải của người dịch: Câu ví dụ và các ký hiệu đặc biệt

Câu ví dụ tiếng Anh "It is raining. I need an umbrella." nghĩa là "Trời đang mưa. Tôi cần một chiếc ô."; "a hat" là "một cái mũ". [CLS] (viết tắt của *classification*) là token đặc biệt luôn đặt ở đầu chuỗi, [SEP] đánh dấu chỗ kết thúc mỗi câu. Trò chơi ở đây giống bài tập "điền từ vào chỗ trống" trong sách tiếng Anh: mô hình nhìn các từ xung quanh ("trời mưa", "tôi cần … ô") để đoán từ bị che. Ví dụ về con số: với một chuỗi 100 token, khoảng 15 token được chọn; trong đó khoảng 12 token bị thay bằng [MASK], khoảng 1–2 token bị thay bằng từ ngẫu nhiên, và khoảng 1–2 token giữ nguyên. Mô hình không biết token nào thuộc loại nào, nên phải "nghi ngờ" mọi vị trí và luôn xây dựng biểu diễn tốt cho từng từ.

Gọi $\mathcal{A}(\mathbf{x})$ là tập các vị trí được chọn trong chuỗi token $\mathbf{x}$, và $\bar{\mathbf{x}}$ là chuỗi đã sửa đổi của $\mathbf{x}$. Hàm mất mát của mô hình hóa ngôn ngữ có che có thể được định nghĩa là:

$$\text{Loss}_{\text{mlm}} \;=\; -\sum_{i\in\mathcal{A}(\mathbf{x})} \log \Pr_i(x_i|\bar{\mathbf{x}}), \tag{7.18}$$

trong đó $\Pr_i(x_i|\bar{\mathbf{x}})$ là xác suất dự đoán đúng x_i tại vị trí i khi biết $\bar{\mathbf{x}}$. Hình 7.5 trình bày một ví dụ minh họa từng bước cách tính Loss_{mlm}.

Chú giải của người dịch: Đọc Phương trình (7.18)

$\Pr_i(x_i|\bar{\mathbf{x}})$ là xác suất mà mô hình gán cho *từ đúng* x_i tại vị trí i sau khi nhìn chuỗi đã bị sửa $\bar{\mathbf{x}}$. Nếu mô hình đoán tốt, xác suất này gần 1 nên log gần 0; nếu đoán kém, xác suất gần 0 nên log là số âm rất lớn. Dấu trừ đổi nó thành "điểm phạt" dương. Ví dụ: tại 4 vị trí được chọn, mô hình gán xác suất cho từ đúng lần lượt là $0.5, 0.8, 0.25, 0.9$; khi đó $\text{Loss}_{\text{mlm}} = -(\ln 0.5 + \ln 0.8 + \ln 0.25 + \ln 0.9) \approx 0.69 + 0.22 + 1.39 + 0.11 = 2.41$. Lưu ý chỉ các vị trí được chọn mới được tính, các vị trí khác không đóng góp vào tổng.

Đối với dự đoán câu tiếp theo, ta làm theo phương pháp đã mô tả trong Mục 7.2.2. Mỗi mẫu huấn luyện được gán một nhãn từ tập $\{\text{IsNext}, \text{NotNext}\}$, ví dụ,

Chuỗi: [CLS] It is raining . [SEP] I need an umbrella . [SEP]
Nhãn: IsNext

Chuỗi: [CLS] The cat sleeps on the windowsill . [SEP] Apples grow on trees . [SEP]
Nhãn: NotNext

[6]Vectơ đầu ra của bộ mã hóa cho token đầu tiên [CLS] được xem là biểu diễn của cả chuỗi, ký hiệu là $\mathbf{h}_{\text{cls}}$ (hoặc $\mathbf{h}_0$). Một bộ phân loại (*classifier*) được xây dựng bên trên $\mathbf{h}_{\text{cls}}$. Khi đó, ta có thể tính xác suất của nhãn c khi biết $\mathbf{h}_{\text{cls}}$, tức là $\Pr(c|\mathbf{h}_{\text{cls}})$. Có nhiều hàm mất mát có thể chọn cho các bài toán phân loại. Ví dụ, trong huấn luyện theo ước lượng hợp lý cực đại (*maximum likelihood*), ta có thể định nghĩa Loss_{nsp} là

$$\text{Loss}_{\text{nsp}} \;=\; -\log \Pr(c_{\text{gold}}|\mathbf{h}_{\text{cls}}), \tag{7.19}$$

[6]**[ND]** Câu thứ hai: "Con mèo ngủ trên bậu cửa sổ. Táo mọc trên cây." – hai câu không liên quan nên có nhãn NotNext (không phải câu tiếp theo); còn "Trời đang mưa. Tôi cần một chiếc ô." là hai câu nối tiếp hợp lý nên có nhãn IsNext (là câu tiếp theo).

trong đó c_{gold} là nhãn đúng của mẫu này.

2. Thiết lập mô hình

Như được minh họa trong Hình 7.6, các mô hình BERT dựa trên kiến trúc bộ mã hóa Transformer chuẩn. Đầu vào là một chuỗi các vectơ nhúng (*embedding*), mỗi vectơ là tổng của nhúng token (*token embedding*), nhúng vị trí (*positional embedding*) và nhúng đoạn (*segment embedding*).

Giải nghĩa các nhãn trong Hình 7.5: Input – đầu vào; *Token Selection* – chọn token ("chọn token với xác suất 15%"); *Token Masking* – che token ("che các token được chọn với xác suất 80%"); *Token Replacement* – thay token ("thay đổi các token được chọn với xác suất 10%"); *Unchanged* – giữ nguyên ("giữ nguyên các token được chọn với xác suất 10%"); dòng cuối: "huấn luyện bộ mã hóa Transformer bằng chuỗi đã sửa đổi". Phía trên, mô hình phải đoán lại các từ gốc *raining*, *I*, *an*, *umbrella* tại các vị trí được chọn.

$$\mathbf{e} \;=\; \mathbf{x} + \mathbf{e}_{\text{pos}} + \mathbf{e}_{\text{seg}}. \tag{7.20}$$

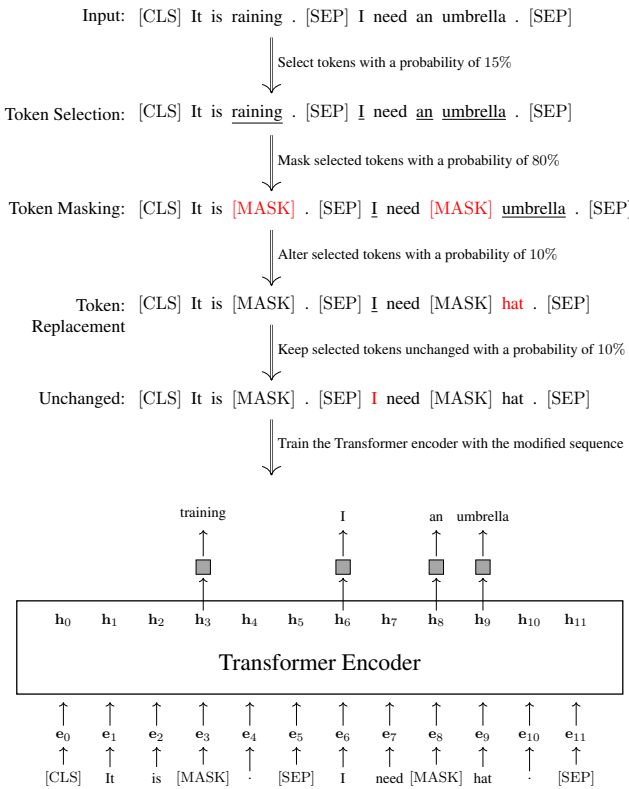
Cả nhúng token ($\mathbf{x}$) và nhúng vị trí ($\mathbf{e}_{\text{pos}}$) đều giống hệt với những gì dùng trong Transformer nguyên bản. Nhúng đoạn ($\mathbf{e}_{\text{seg}}$) là thành phần mới được bổ sung, cho biết một token thuộc về Sent_A hay Sent_B. Điều này được minh họa qua ví dụ sau:

Token	[CLS]	It	is	raining	.	[SEP]	I	need	an	umbrella	.	[SEP]
$\mathbf{x}$	$\mathbf{x}_0$	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$	$\mathbf{x}_6$	$\mathbf{x}_7$	$\mathbf{x}_8$	$\mathbf{x}_9$	$\mathbf{x}_{10}$	$\mathbf{x}_{11}$
$\mathbf{e}_{\text{pos}}$	PE(0)	PE(1)	PE(2)	PE(3)	PE(4)	PE(5)	PE(6)	PE(7)	PE(8)	PE(9)	PE(10)	PE(11)
$\mathbf{e}_{\text{seg}}$	$\mathbf{e}_A$	$\mathbf{e}_A$	$\mathbf{e}_A$	$\mathbf{e}_A$	$\mathbf{e}_A$	$\mathbf{e}_A$	$\mathbf{e}_B$	$\mathbf{e}_B$	$\mathbf{e}_B$	$\mathbf{e}_B$	$\mathbf{e}_B$	$\mathbf{e}_B$

Chú giải của người dịch: Cộng ba vectơ nhúng

Mỗi token được mô tả bằng ba "thẻ thông tin", mỗi thẻ là một vectơ cùng số chiều: *nó là từ gì* ($\mathbf{x}$), *nó đứng ở vị trí thứ mấy* ($\mathbf{e}_{\text{pos}}$, ở đây $\text{PE}(i)$ là mã hóa vị trí của vị trí i), và *nó thuộc câu A hay câu B* ($\mathbf{e}_{\text{seg}}$, chỉ có hai giá trị $\mathbf{e}_A, \mathbf{e}_B$). Ba vectơ được cộng theo từng tọa độ như cộng vectơ ở lớp 10. Ví dụ thu nhỏ với 3 chiều: $\mathbf{x} = (0.2;\ 1.0;\ -0.5)$, $\mathbf{e}_{\text{pos}} = (0.1;\ 0;\ 0.3)$, $\mathbf{e}_{\text{seg}} = (0;\ -0.2;\ 0.1)$ cho $\mathbf{e} = (0.3;\ 0.8;\ -0.1)$. Cùng một từ "I" đặt ở câu A hay câu B, ở vị trí khác nhau, sẽ cho vectơ $\mathbf{e}$ khác nhau.

Phần chính của các mô hình BERT là một mạng Transformer nhiều



Hình 7.5: Một ví dụ minh họa từng bước cho mô hình hóa ngôn ngữ có che kiểu BERT. Trước hết, 15% số token được chọn ngẫu nhiên. Các token được chọn này sau đó được xử lý theo một trong ba cách: thay bằng token [MASK] (80% số lần), thay bằng một token ngẫu nhiên (10% số lần), hoặc giữ nguyên (10% số lần). Mô hình được huấn luyện để dự đoán các token được chọn này dựa trên chuỗi đã sửa đổi. $\mathbf{e}_i$ biểu diễn vectơ nhúng của token tại vị trí i. Các ô màu xám biểu diễn các tầng Softmax.

tầng. Một tầng Transformer gồm một tầng con (*sub-layer*) tự chú ý và một tầng con FFN. Cả hai tầng con đều theo kiến trúc chuẩn hóa sau (*post-norm*): output $=$ LNorm$(F(\text{input}) + \text{input})$, trong đó $F(\cdot)$ là hàm lõi của tầng con (hoặc là một mô hình tự chú ý (*self-attention*), hoặc là một FFN), và LNorm$(\cdot)$ là đơn vị chuẩn hóa tầng (*layer normalization*). Thông thường, nhiều tầng Transformer được xếp chồng lên nhau để tạo thành một mạng sâu. Với mỗi vị trí trong chuỗi, biểu diễn đầu ra cuối cùng là vectơ số thực do tầng trên cùng sinh ra.

Có một số khía cạnh mà ta có thể cân nhắc khi phát triển các mô hình BERT.

- **Kích thước bộ từ vựng** ($|V|$). Trong Transformer, mỗi token tương ứng với một mục trong bộ từ vựng (*vocabulary*) V. Bộ từ vựng lớn có thể bao phủ nhiều biến thể hình thức bề mặt của từ hơn, nhưng có thể dẫn tới yêu cầu lưu trữ tăng lên.

- **Kích thước nhúng** (d_e). Mỗi token được biểu diễn bằng một vectơ số thực d_e chiều. Như đã trình bày ở trên, vectơ này là tổng của nhúng token, nhúng vị trí và nhúng đoạn, tất cả đều cũng là các vectơ số thực d_e chiều.

- **Kích thước ẩn** (d). Đầu vào và đầu ra của một tầng con có d chiều. Ngoài ra, phần lớn các trạng thái ẩn (*hidden state*) của một tầng con là các vectơ d chiều. Nói chung, d có thể được xem gần đúng là độ rộng của mạng.

- **Số đầu chú ý** (n_{head}). Trong các tầng con tự chú ý, ta cần chỉ định số đầu dùng trong tự chú ý đa đầu (*multi-head self-attention*). Số đầu lớn hơn cho phép mô hình đồng thời chú ý tới thông tin từ nhiều không gian con biểu diễn (*representation subspaces*) hơn. Trong các hệ thống thực tế, ta thường đặt $n_{\text{head}} \geq 4$.

- **Kích thước ẩn của FFN** (d_{ffn}). Kích thước tầng ẩn của các FFN dùng trong Transformer thường lớn hơn d. Ví dụ, một thiết lập điển hình là $d_{\text{ffn}} = 4d$. Với các Transformer lớn hơn, chẳng hạn các mô hình lớn gần đây, d_{ffn} có thể được đặt ở một giá trị rất lớn.

- **Độ sâu mô hình** (L). Dùng mạng sâu là một cách hiệu quả để cải thiện năng lực biểu đạt (*expressive power*) của Transformer. Với các mô hình BERT, L thường được đặt là 12 hoặc 24. Tuy nhiên, các

mạng còn sâu hơn nữa cũng khả thi và có thể được áp dụng để cải thiện thêm.

Các thiết lập khác nhau của những siêu tham số (*hyper-parameter*) này dẫn tới các kích thước mô hình khác nhau. Hai cấu hình được dùng rộng rãi là $\text{BERT}_{\text{base}}$ và $\text{BERT}_{\text{large}}$:

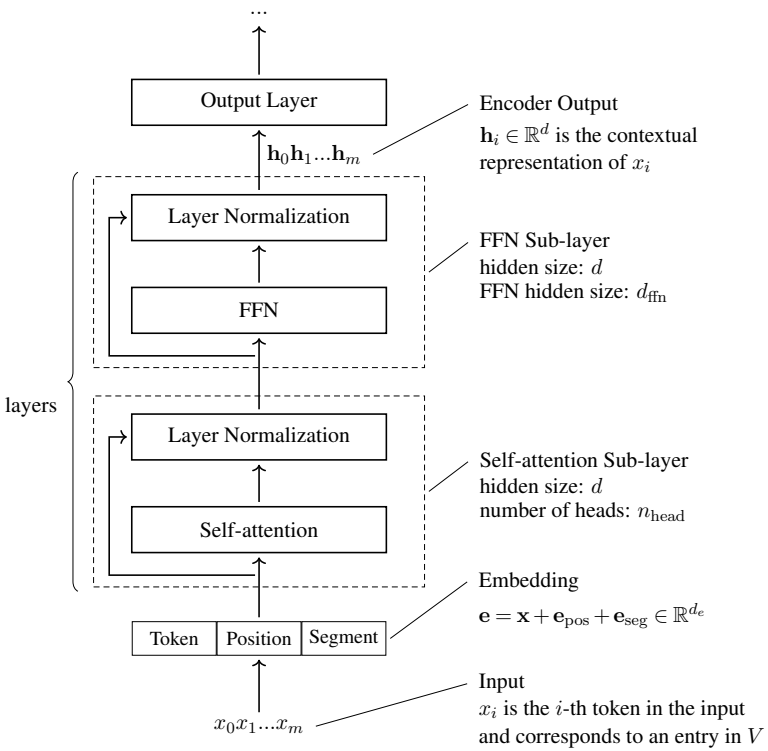
- $\text{BERT}_{\text{base}}$: $d = 768$, $L = 12$, $n_{\text{head}} = 12$, tổng số tham số $= 110$M.
- $\text{BERT}_{\text{large}}$: $d = 1{,}024$, $L = 24$, $n_{\text{head}} = 16$, tổng số tham số $=$ 340M.

Giải nghĩa các nhãn trong Hình 7.6: Input – đầu vào (x_i là token thứ i của đầu vào, ứng với một mục trong V); *Embedding* – vectơ nhúng (*Token/Position/Segment*: nhúng token/vị trí/đoạn); *Self-attention Sub-layer* – tầng con tự chú ý (*hidden size*: kích thước ẩn, *number of heads*: số đầu); *FFN Sub-layer* – tầng con FFN (*FFN hidden size*: kích thước ẩn của FFN); *Layer Normalization* – chuẩn hóa tầng; *layers* – các tầng (khối được lặp lại L lần); *Output Layer* – tầng đầu ra; *Encoder Output* – đầu ra bộ mã hóa ($\mathbf{h}_i \in \mathbb{R}^d$ là biểu diễn theo ngữ cảnh của x_i).

Chú giải của người dịch: Hình dung các con số 110M, 340M

"M" là triệu (*million*): $\text{BERT}_{\text{base}}$ có khoảng 110 triệu tham số, tức 110 triệu con số thực cần học. Phần lớn tham số nằm trong các ma trận trọng số của các tầng. Ước lượng thô: mỗi tầng Transformer có khoảng $12d^2$ tham số (khoảng $4d^2$ cho tự chú ý và $8d^2$ cho FFN khi $d_{\text{ffn}} = 4d$). Với $d = 768$, $L = 12$: $12 \times 12 \times 768^2 \approx 85$ triệu; cộng thêm bảng nhúng token (khoảng 30 000 từ $\times 768 \approx 23$ triệu) là xấp xỉ 110 triệu. Làm tương tự với $d = 1024$, $L = 24$ cho khoảng 300 triệu, cộng bảng nhúng ra gần 340 triệu.

Việc huấn luyện các mô hình BERT tuân theo quy trình chuẩn cho Transformer. Huấn luyện các mô hình lớn hơn như $\text{BERT}_{\text{large}}$ đòi hỏi nhiều công sức và thời gian huấn luyện hơn. Đây là vấn đề phổ biến của tiền huấn luyện, đặc biệt khi mô hình được huấn luyện trên một lượng dữ liệu rất lớn. Trong thực tế, hiệu quả huấn luyện thường là mối quan tâm lớn. Một cách làm phổ biến là trước hết huấn luyện mô hình BERT trên các chuỗi tương đối ngắn trong một số lượng lớn bước huấn luyện, rồi tiếp tục huấn luyện nó trên các chuỗi có độ dài đầy đủ trong các bước huấn luyện còn lại.



Hình 7.6: Kiến trúc mô hình của BERT (bộ mã hóa Transformer). Các token đầu vào trước hết được biểu diễn thành các vectơ nhúng, mỗi vectơ là tổng của nhúng token, nhúng vị trí và nhúng đoạn tương ứng. Sau đó, chuỗi vectơ nhúng được xử lý bởi một chồng các tầng Transformer. Mỗi tầng trong chồng này gồm một tầng con tự chú ý và một tầng con FFN. Đầu ra của mô hình BERT là một chuỗi vectơ do tầng Transformer cuối cùng sinh ra.

7.3.2 Huấn luyện nhiều hơn và mô hình lớn hơn

BERT là một cột mốc trong NLP và đã khơi nguồn cho rất nhiều công trình tiếp theo. Một hướng là mở rộng quy mô tiền huấn luyện, thường gồm hai cách: tăng lượng dữ liệu huấn luyện và tăng số tham số của mô hình.

RoBERTa, một mở rộng của mô hình BERT chuẩn, là một ví dụ cho những nỗ lực như vậy [Liu và cộng sự, 2019]. Nó đưa ra hai cải tiến chính. Thứ nhất, chỉ cần dùng nhiều dữ liệu huấn luyện hơn và nhiều tài nguyên tính toán hơn là có thể cải thiện mô hình BERT mà không cần thay đổi kiến trúc mô hình. Thứ hai, việc bỏ hàm mất mát NSP không làm giảm hiệu năng trên các nhiệm vụ hạ nguồn nếu quy mô huấn luyện được mở rộng. Những phát hiện này gợi ý một hướng khám phá chung cho tiền huấn luyện: ta có thể tiếp tục cải thiện tiền huấn luyện bằng cách mở rộng quy mô trên những nhiệm vụ tiền huấn luyện đơn giản.

Cách thứ hai để cải thiện mô hình BERT là tăng số tham số của mô hình. Ví dụ, trong công trình của He và cộng sự [2021], một mô hình tương tự BERT với 1,5 tỉ tham số được xây dựng bằng cách tăng cả độ sâu mô hình lẫn kích thước ẩn. Tuy nhiên, việc mở rộng quy mô BERT và nhiều mô hình tiền huấn luyện khác đặt ra những thách thức mới trong huấn luyện, chẳng hạn huấn luyện các mô hình rất lớn thường trở nên không ổn định và khó hội tụ. Điều này làm bài toán phức tạp hơn và đòi hỏi cân nhắc cẩn thận nhiều khía cạnh, như kiến trúc mô hình, tính toán song song (*parallel computation*) và khởi tạo tham số (*parameter initialization*). Trong một ví dụ khác, Shoeybi và cộng sự [2019] đã huấn luyện thành công một mô hình tương tự BERT với 3,9 tỉ tham số, trong đó hàng trăm GPU được dùng để đáp ứng nhu cầu tính toán tăng lên.

Chú giải của người dịch: GPU và "khó hội tụ"

GPU (*graphics processing unit*, bộ xử lý đồ họa) vốn là chip dùng để vẽ hình trong trò chơi điện tử, nhưng rất giỏi làm hàng nghìn phép nhân cùng lúc nên trở thành "động cơ" chính để huấn luyện mạng nơ-ron. "Hội tụ" nghĩa là hàm mất mát giảm dần rồi ổn định ở một giá trị thấp, giống dãy số có giới hạn mà bạn học ở lớp 11. Mô hình quá lớn có thể "không ổn định": hàm mất mát đột ngột tăng vọt hoặc dao động mạnh thay vì giảm đều.

7.3.3 Các mô hình hiệu quả hơn

So với các mô hình đi trước, BERT là một mô hình tương đối lớn vào thời điểm nó được đề xuất. Kích thước mô hình tăng lên dẫn đến yêu cầu bộ nhớ lớn hơn và suy luận (*inference*) chậm hơn. Việc phát triển các mô hình BERT nhỏ hơn và nhanh hơn là một phần của thách thức rộng hơn về xây dựng các Transformer hiệu quả, vốn đã được thảo luận kỹ trong Chương 6. Tuy nhiên, việc đi sâu vào chủ đề này nằm ngoài phạm vi của mục này. Ở đây, ta chỉ tập trung vào một vài biến thể hiệu quả của BERT.

Có một số hướng nghiên cứu đặc biệt đáng quan tâm để phát triển các mô hình BERT hiệu quả. Thứ nhất, các công trình về chưng cất tri thức (*knowledge distillation*), chẳng hạn huấn luyện một mô hình học trò (*student model*) bằng đầu ra của một mô hình thầy (*teacher model*) đã được huấn luyện tốt, cho thấy có thể thu được những mô hình gọn hơn bằng cách chuyển giao tri thức từ các mô hình lớn hơn. Vì mô hình BERT là mạng nhiều tầng với một số loại tầng khác nhau, chưng cất tri thức có thể được áp dụng ở nhiều cấp độ biểu diễn khác nhau. Ví dụ, ngoài việc chưng cất tri thức từ các tầng đầu ra, ta cũng có thể đưa vào một hàm mất mát huấn luyện đo sự khác biệt giữa đầu ra của các tầng ẩn của mô hình thầy và mô hình học trò [Sun và cộng sự, 2020b; Jiao và cộng sự, 2020]. Thực tế, chưng cất tri thức đã là một trong những kỹ thuật được dùng rộng rãi nhất để học các mô hình tiền huấn luyện nhỏ.

Chú giải của người dịch: Chưng cất tri thức

Hãy hình dung một thầy giáo giỏi (mô hình lớn) và một học trò (mô hình nhỏ). Thay vì chỉ cho học trò xem đáp án đúng ("nhãn cứng"), thầy cho xem cả mức độ tự tin của mình với từng phương án, ví dụ "70% là A, 25% là B, 5% là C". Thông tin chi tiết này giúp học trò học nhanh và tốt hơn. Học trò được huấn luyện để đầu ra của mình gần với đầu ra của thầy; nhờ đó mô hình nhỏ đạt được phần lớn năng lực của mô hình lớn nhưng chạy nhanh hơn nhiều.

Thứ hai, các phương pháp nén mô hình (*model compression*) truyền thống có thể được áp dụng trực tiếp cho BERT. Một cách phổ biến là áp dụng các phương pháp tỉa (*pruning*) đa dụng cho các bộ mã hóa Transformer [Gale và cộng sự, 2019]. Việc này thường bao gồm loại bỏ toàn bộ một số tầng [Fan và cộng sự, 2019] hoặc một tỉ lệ phần trăm nhất

định các tham số trong mạng [Sanh và cộng sự, 2020; Chen và cộng sự, 2020b]. Tỉa cũng áp dụng được cho các mô hình chú ý đa đầu (*multi-head attention*). Ví dụ, Michel và cộng sự [2019] chỉ ra rằng việc loại bỏ một số đầu chú ý không làm giảm đáng kể hiệu năng của mô hình BERT, mà lại tăng tốc suy luận của các mô hình này. Một cách khác để nén mô hình BERT là lượng tử hóa (*quantization*) [Shen và cộng sự, 2020b]. Bằng cách biểu diễn các tham số mô hình dưới dạng số có độ chính xác thấp (*low-precision*), ta có thể nén mô hình đáng kể. Mặc dù phương pháp này không dành riêng cho mô hình BERT, nó tỏ ra hiệu quả với các kiến trúc lớn dựa trên Transformer.

Chú giải của người dịch: Lượng tử hóa

Mỗi tham số thường được lưu bằng một số thực 32 bit (4 byte). Lượng tử hóa giống việc làm tròn số: thay vì lưu $0.73215\ldots$ với độ chính xác cao, ta chỉ lưu một số nguyên 8 bit (từ -128 đến 127) cùng một hệ số tỉ lệ chung. Ví dụ, nếu hệ số tỉ lệ là 0.01 thì 0.73215 được lưu thành số nguyên 73 (giá trị khôi phục là 0.73). Bộ nhớ giảm khoảng 4 lần (110 triệu tham số: từ khoảng 440 MB xuống khoảng 110 MB), đổi lại sai số làm tròn nhỏ.

Thứ ba, vì mô hình BERT là những mạng tương đối sâu và lớn, một hướng nghiên cứu khác sử dụng mạng động (*dynamic network*) để điều chỉnh các mô hình này cho suy luận hiệu quả. Một ý tưởng trong khuôn khổ này là chọn động các tầng dùng để xử lý một token; ví dụ, trong các mô hình thích ứng độ sâu (*depth-adaptive*), ta thoát ra ở một độ sâu tối ưu nào đó và do đó bỏ qua các tầng còn lại trong chồng tầng [Xin và cộng sự, 2020; Zhou và cộng sự, 2020]. Tương tự, ta có thể phát triển các mô hình thích ứng độ dài (*length-adaptive*), trong đó độ dài của chuỗi đầu vào được điều chỉnh động. Ví dụ, ta có thể bỏ qua một số token trong chuỗi đầu vào để mô hình giảm khối lượng tính toán cho những token ít quan trọng, qua đó nâng cao hiệu quả tổng thể.

Thứ tư, ta cũng có thể chia sẻ tham số (*parameter sharing*) giữa các tầng để giảm kích thước mô hình BERT. Một cách đơn giản để làm điều này là chia sẻ tham số của cả một tầng Transformer cho toàn bộ chồng tầng [Dehghani và cộng sự, 2018; Lan và cộng sự, 2020]. Ngoài việc giảm số tham số, điều này cho phép tái sử dụng cùng một tầng trong mạng Transformer nhiều tầng, giúp tiết kiệm bộ nhớ khi chạy kiểm tra.

7.3.4 Các mô hình đa ngôn ngữ

Mô hình BERT ban đầu chủ yếu tập trung vào tiếng Anh. Ngay sau khi mô hình này được đề xuất, nó đã được mở rộng sang nhiều ngôn ngữ. Một cách đơn giản là phát triển một mô hình riêng cho mỗi ngôn ngữ. Một cách khác, đã trở nên phổ biến hơn trong các công trình gần đây về mô hình ngôn ngữ lớn (*large language model*), là huấn luyện trực tiếp các mô hình đa ngôn ngữ (*multilingual*) trên dữ liệu của tất cả các ngôn ngữ. Theo đó, các mô hình BERT đa ngôn ngữ (mBERT) đã được phát triển bằng cách huấn luyện trên văn bản của 104 ngôn ngữ[7]. Khác biệt chính so với các mô hình BERT đơn ngữ (*monolingual*) là mô hình mBERT dùng bộ từ vựng lớn hơn để bao phủ token của nhiều ngôn ngữ. Nhờ vậy, biểu diễn của các token thuộc các ngôn ngữ khác nhau được ánh xạ vào cùng một không gian, cho phép chia sẻ tri thức giữa các ngôn ngữ.

Một trong những ứng dụng quan trọng nhất của các mô hình tiền huấn luyện đa ngôn ngữ là học chuyển giao xuyên ngôn ngữ (*cross-lingual transfer learning*). Trong thiết lập xuyên ngôn ngữ zero-shot (*zero-shot cross-lingual*), một mô hình được tinh chỉnh cho một nhiệm vụ bằng dữ liệu của một ngôn ngữ nguồn (*source language*) rồi được áp dụng cho cùng nhiệm vụ đó ở một ngôn ngữ đích (*target language*) mà không cần bất kỳ dữ liệu huấn luyện nào của ngôn ngữ đích. Chẳng hạn, trong phân loại văn bản (*text classification*) xuyên ngôn ngữ, ta có thể tinh chỉnh một mô hình tiền huấn luyện đa ngôn ngữ trên các tài liệu tiếng Anh đã gán nhãn, rồi dùng trực tiếp mô hình đã tinh chỉnh để phân loại các tài liệu tiếng Trung.

Chú giải của người dịch: Chuyển giao xuyên ngôn ngữ và tiếng Việt

Điều này đặc biệt có ích cho tiếng Việt: dữ liệu gán nhãn tiếng Việt cho nhiều nhiệm vụ còn ít, trong khi tiếng Anh có rất nhiều. Ta có thể tinh chỉnh một mô hình đa ngôn ngữ (ví dụ mBERT, vốn có tiếng Việt trong số các ngôn ngữ huấn luyện) để phân loại cảm xúc bình luận tiếng Anh, rồi đem dùng ngay cho bình luận tiếng Việt. Điều này khả thi vì trong không gian biểu diễn chung, câu "I love this movie" và "Tôi rất thích bộ phim này" có xu hướng được biểu diễn bằng những vectơ gần nhau.

[7] `https://github.com/google-research/bert/`

Một cải tiến cho các mô hình tiền huấn luyện đa ngôn ngữ như mBERT là đưa dữ liệu song ngữ (*bilingual data*) vào tiền huấn luyện. Thay vì chỉ huấn luyện trên dữ liệu đơn ngữ của nhiều ngôn ngữ, huấn luyện song ngữ mô hình hóa một cách tường minh mối quan hệ giữa các token trong hai ngôn ngữ. Mô hình thu được sẽ có khả năng chuyển giao xuyên ngôn ngữ, và do đó có thể dễ dàng thích ứng với các ngôn ngữ khác nhau. Lample và Conneau [2019] đề xuất một cách tiếp cận để tiền huấn luyện **mô hình ngôn ngữ xuyên ngôn ngữ** (*cross-lingual language models*) (**XLM**). Trong công trình của họ, một mô hình ngôn ngữ xuyên ngôn ngữ có thể được huấn luyện bằng mô hình hóa ngôn ngữ nhân quả (*causal language modeling*) hoặc mô hình hóa ngôn ngữ có che. Với tiền huấn luyện bằng mô hình hóa ngôn ngữ có che, mô hình được xem là một bộ mã hóa. Mục tiêu huấn luyện giống như BERT: ta cực đại hóa xác suất của một số token được chọn ngẫu nhiên, vốn hoặc bị che, hoặc bị thay bằng token ngẫu nhiên, hoặc được giữ nguyên trong đầu vào. Nếu xét dữ liệu song ngữ trong tiền huấn luyện, mỗi lần ta lấy mẫu một cặp câu đã được căn chỉnh (*aligned*) với nhau. Sau đó, hai câu được ghép lại thành một chuỗi duy nhất dùng để huấn luyện. Ví dụ, xét một cặp câu Anh–Trung

鯨鱼 是 哺乳 动物 。 $\leftrightarrow$ Whales are mammals .

Ta có thể ghép chúng lại để được một chuỗi như sau

[CLS] 鯨鱼 是 哺乳 动物 。 [SEP] Whales are mammals . [SEP]

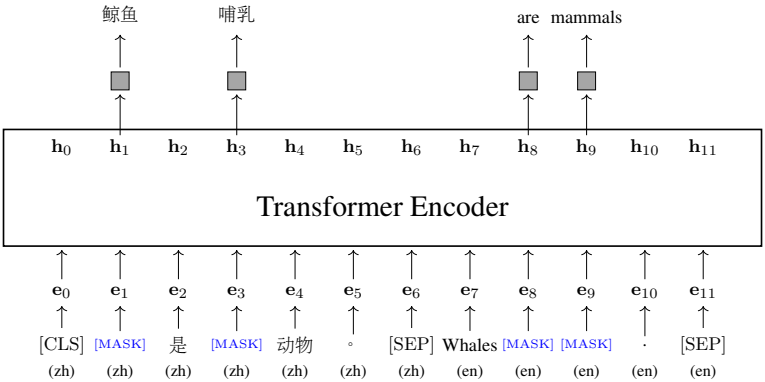
Sau đó ta chọn một tỉ lệ phần trăm nhất định các token và thay chúng bằng [MASK].

[CLS] [MASK] 是 [MASK] 动物 。 [SEP] Whales [MASK] [MASK] . [SEP]

[8]Mục tiêu của tiền huấn luyện là cực đại hóa tích các xác suất của những token bị che khi biết chuỗi trên. Bằng cách huấn luyện như vậy, mô hình có thể học cách biểu diễn cả chuỗi tiếng Anh lẫn chuỗi tiếng Trung, đồng thời nắm bắt được sự tương ứng giữa các token trong hai ngôn ngữ. Ví dụ, dự đoán token tiếng Trung 鯨鱼 có thể cần thông tin từ token tiếng

[8]**[ND]** Câu ví dụ nghĩa là "Cá voi là động vật có vú." Trong câu tiếng Trung, 鯨鱼 = cá voi, 是 = là, 哺乳 = (cho) bú, 动物 = động vật.

Anh *Whales*. Việc căn chỉnh biểu diễn của hai ngôn ngữ về bản chất biến mô hình thành một mô hình "dịch". Vì vậy, mục tiêu huấn luyện này còn được gọi là **mô hình hóa ngôn ngữ dịch** (*translation language modeling*). Hình 7.7 minh họa cách tiếp cận này.



Hình 7.7: Minh họa mô hình hóa ngôn ngữ dịch (TLM). Để dễ hiểu, chúng tôi trình bày một ví dụ đơn giản hóa trong đó mọi token được chọn đều bị che. Mô hình được huấn luyện để dự đoán các token bị che này. Vì chuỗi đầu vào chứa các token song song của hai ngôn ngữ, việc dự đoán một token bị che ở ngôn ngữ này cho phép mô hình chú ý tới ngữ cảnh ở ngôn ngữ kia, qua đó buộc mô hình phải căn chỉnh xuyên ngôn ngữ một cách tường minh. Trong cách xây dựng XLM nguyên bản của Lample và Conneau [2019], vectơ nhúng đầu vào (tức $\mathbf{e}_i$) là tổng của nhúng token, nhúng vị trí và một nhúng ngôn ngữ (*language embedding*) riêng. Điều này đòi hỏi mỗi token phải được gán một nhãn ngôn ngữ, cho phép mô hình phân biệt giữa các ngôn ngữ. Tuy nhiên, trong các kiến trúc tiền huấn luyện đa ngôn ngữ hiện đại (như XLM-R), việc chỉ rõ ngôn ngữ là không cần thiết, vì chỉ cần một bộ từ vựng chung là đủ. Hơn nữa, việc dựa vào nhúng ngôn ngữ khiến khó xử lý hiện tượng chuyển mã. Do đó, ở đây ta giả định rằng mọi biểu diễn token đều không phụ thuộc ngôn ngữ (*language-agnostic*).

Ghi chú Hình 7.7: (zh) là tiếng Trung, (en) là tiếng Anh; mô hình phải đoán lại 鯨鱼 (cá voi), 哺乳 (cho bú), *are* (là), *mammals* (động vật có vú).

Một lợi ích của các mô hình tiền huấn luyện đa ngôn ngữ là khả năng vốn có trong việc xử lý chuyển mã (*code-switching*). Trong NLP và ngôn ngữ học, chuyển mã là việc chuyển đổi qua lại giữa các ngôn ngữ trong

một văn bản. Ví dụ, sau đây là một văn bản hỗn hợp chứa cả tiếng Trung và tiếng Anh:

周末 我们 打算 去 做 hiking , 你 想 一起 来 吗 ?

(Cuối tuần chúng tôi định đi *hiking* (đi bộ đường dài), bạn có muốn đi cùng không?)

Với các mô hình tiền huấn luyện đa ngôn ngữ, ta không cần xác định một token là tiếng Trung hay tiếng Anh. Thay vào đó, mỗi token chỉ là một mục trong bộ từ vựng chung. Có thể hình dung điều này như việc tạo ra một ngôn ngữ "mới" bao gồm tất cả các ngôn ngữ mà ta muốn xử lý.

Chú giải của người dịch: Chuyển mã quanh ta

Chuyển mã rất quen thuộc với người Việt trẻ, chẳng hạn "Deadline tuần này căng quá, mình phải check lại slide trước khi meeting". Một mô hình đa ngôn ngữ coi "deadline", "check" hay "căng" đều chỉ là các mục trong cùng một bộ từ vựng, nên đọc được những câu như vậy mà không cần biết trước từ nào thuộc ngôn ngữ nào.

Kết quả của tiền huấn luyện đa ngôn ngữ chịu ảnh hưởng của nhiều yếu tố. Khi kiến trúc mô hình đã cố định, ta cần chỉ định nhiều cấu hình, chẳng hạn kích thước bộ từ vựng chung, số lượng (hoặc tỉ lệ) mẫu của mỗi ngôn ngữ và kích thước mô hình. Conneau và cộng sự [2020] chỉ ra một số vấn đề thú vị liên quan đến tiền huấn luyện đa ngôn ngữ quy mô lớn cho các mô hình kiểu XLM. Thứ nhất, khi số ngôn ngữ được hỗ trợ tăng lên, cần một mô hình lớn hơn để xử lý các ngôn ngữ này. Thứ hai, một bộ từ vựng chung lớn hơn giúp mô hình hóa sự đa dạng tăng lên của các ngôn ngữ. Thứ ba, các ngôn ngữ ít tài nguyên (*low-resource languages*) được hưởng lợi đáng kể từ chuyển giao xuyên ngôn ngữ, đặc biệt khi kho ngữ liệu tiền huấn luyện có các ngôn ngữ nhiều tài nguyên (*high-resource languages*) tương đồng về mặt ngôn ngữ học. Tuy nhiên, một hiện tượng được gọi là **lời nguyền đa ngôn ngữ** (*the curse of multilinguality*) (hay giao thoa năng lực (*capacity interference*)) xuất hiện khi một mô hình có năng lực cố định được huấn luyện trên quá nhiều ngôn ngữ: việc thêm nhiều ngôn ngữ hơn cuối cùng sẽ buộc các ngôn ngữ phải cạnh tranh tham số, làm giảm hiệu năng trên các ngôn ngữ nhiều tài nguyên. Vì vậy, trong các hệ thống thực tế, có một sự đánh đổi giữa số ngôn ngữ được hỗ trợ và năng lực của mô hình.

Chú giải của người dịch: Lời nguyền đa ngôn ngữ

Hãy hình dung mô hình như một chiếc ba lô có dung tích cố định (số tham số cố định). Mỗi ngôn ngữ cần một chỗ để "cất" kiến thức. Khi chỉ có vài ngôn ngữ, chúng còn giúp nhau (ví dụ tiếng Lào học được nhiều từ tiếng Thái gần gũi). Nhưng khi nhồi quá nhiều ngôn ngữ vào cùng một ba lô, mỗi ngôn ngữ chỉ còn ít chỗ, và ngay cả những ngôn ngữ vốn mạnh như tiếng Anh cũng bị "chật" và làm kém đi.

7.4 Ứng dụng các mô hình BERT

Sau khi được tiền huấn luyện, một mô hình BERT có thể được dùng để giải các bài toán NLP. Nhưng các mô hình BERT chưa sẵn sàng ngay cho các nhiệm vụ hạ nguồn. Nói chung, cần tinh chỉnh thêm để thích ứng chúng. Bước đầu tiên, ta cần một bộ dự đoán (*predictor*) để khớp đầu ra của mô hình với nhiệm vụ cần giải. Gọi $\text{BERT}_{\hat{\theta}}(\cdot)$ là một mô hình BERT với các tham số tiền huấn luyện $\hat{\theta}$, và $\text{Predict}_{\omega}(\cdot)$ là một mạng dự đoán (*prediction network*) với tham số ω. Bằng cách gắn mạng dự đoán vào đầu ra của mô hình BERT, ta xây dựng được một mô hình để giải các nhiệm vụ hạ nguồn. Mô hình này có thể được biểu diễn là:

$$\mathbf{y} = \text{Predict}_{\omega}(\text{BERT}_{\hat{\theta}}(\mathbf{x})), \tag{7.21}$$

trong đó $\mathbf{x}$ là đầu vào và $\mathbf{y}$ là đầu ra phù hợp với bài toán. Ví dụ, trong các bài toán phân loại, mô hình đưa ra một phân phối xác suất (*probability distribution*) trên các nhãn.

Sau đó, ta thu thập một tập các mẫu có nhãn $\mathcal{D}$ và tinh chỉnh mô hình bằng

$$(\tilde{\omega}, \tilde{\theta}) = \arg\min_{\omega, \hat{\theta}^+} \sum_{(\mathbf{x}, \mathbf{y}_{\text{gold}}) \in \mathcal{D}} \text{Loss}(\mathbf{y}_{\omega, \hat{\theta}^+}, \mathbf{y}_{\text{gold}}), \tag{7.22}$$

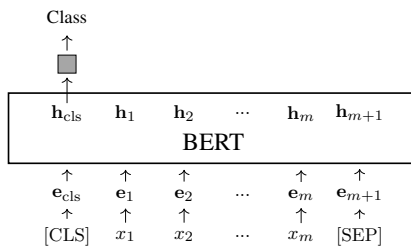
trong đó $(\mathbf{x}, \mathbf{y}_{\text{gold}})$ biểu diễn một bộ gồm một đầu vào và đầu ra tương ứng của nó. Ký hiệu của phương trình này có vẻ hơi phức tạp, nhưng quá trình huấn luyện/tinh chỉnh là chuẩn. Ta tối ưu mô hình bằng cách cực tiểu hóa hàm mất mát trên dữ liệu huấn luyện. Kết quả là các tham số đã tối ưu $\tilde{\omega}$ và $\tilde{\theta}$. Quá trình tối ưu bắt đầu từ các tham số tiền huấn luyện $\hat{\theta}$. Ở đây ta dùng $\hat{\theta}^+$ để chỉ rằng các tham số được khởi tạo bằng $\hat{\theta}$, và dùng $\mathbf{y}_{\omega, \hat{\theta}^+}$ để ký hiệu đầu ra của mô hình được tính bằng các tham số ω và $\hat{\theta}^+$.

Chú giải của người dịch: Đọc các ký hiệu $\hat{\theta}$, $\hat{\theta}^+$, $\tilde{\theta}$

Có thể hiểu theo ba "thời điểm" của bộ tham số BERT: $\hat{\theta}$ ("mũ") là tham số *sau tiền huấn luyện*; $\hat{\theta}^+$ là tham số *đang được tinh chỉnh*, xuất phát từ $\hat{\theta}$; còn $\tilde{\theta}$ ("ngã") là tham số *sau tinh chỉnh*. Tương tự ω là tham số của mạng dự đoán nhỏ gắn thêm, $\tilde{\omega}$ là giá trị sau khi học xong. arg min đọc là "giá trị của biến làm cho biểu thức nhỏ nhất"; ví dụ $\arg\min_x (x-3)^2 = 3$. Như vậy, Phương trình (7.22) nói: tìm bộ tham số làm tổng điểm phạt trên toàn bộ dữ liệu có nhãn $\mathcal{D}$ là nhỏ nhất. Điểm khác so với huấn luyện từ đầu chỉ là ta *xuất phát* từ một mô hình đã "biết" ngôn ngữ, nên chỉ cần ít dữ liệu có nhãn.

Với các tham số đã tinh chỉnh $\tilde{\omega}$ và $\tilde{\theta}$, ta có thể áp dụng mô hình $\text{Predict}_{\tilde{\omega}}(\text{BERT}_{\tilde{\theta}}(\cdot))$ cho dữ liệu mới của chính những nhiệm vụ mà mô hình đã được tinh chỉnh. Dạng của nhiệm vụ hạ nguồn quyết định định dạng đầu vào và đầu ra của mô hình, cũng như kiến trúc của mạng dự đoán. Dưới đây, ta liệt kê một số nhiệm vụ mà mô hình BERT rất phù hợp.

- **Phân loại** (văn bản đơn). Một trong những ứng dụng phổ biến nhất của mô hình BERT là phân loại văn bản. Trong nhiệm vụ này, mô hình BERT nhận một chuỗi token và mã hóa nó thành một chuỗi vectơ. Vectơ đầu ra đầu tiên $\mathbf{h}_{\text{cls}}$ (hoặc $\mathbf{h}_0$) thường được dùng làm biểu diễn của toàn bộ văn bản. Mạng dự đoán nhận $\mathbf{h}_{\text{cls}}$ làm đầu vào để sinh ra một phân phối trên các nhãn. Gọi $[\text{CLS}]\, x_1 x_2 \cdots x_m$ là chuỗi đầu vào. Xem Hình 7.8 để thấy minh họa phân loại văn bản dựa trên BERT. Nhiều bài toán NLP có thể được quy về các



Hình 7.8: Phân loại dựa trên BERT cho một văn bản đơn. Ở đây ô màu xám biểu thị mạng dự đoán. (*Class*: lớp/nhãn dự đoán.)

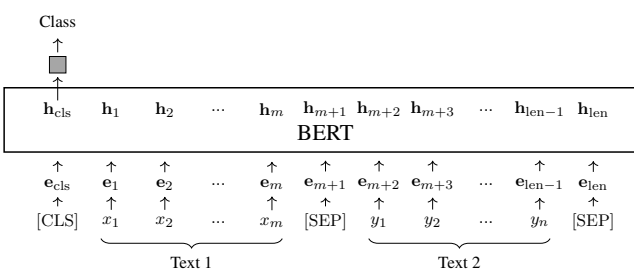
nhiệm vụ phân loại văn bản, và đã có một số bộ đánh giá chuẩn

(*benchmark*) phân loại văn bản để đánh giá các mô hình tiền huấn luyện. Ví dụ, ta có thể phân loại văn bản theo tính đúng ngữ pháp (*grammaticality*) hoặc sắc thái cảm xúc (*sentiment*) [Socher và cộng sự, 2013; Warstadt và cộng sự, 2019]. Lưu ý rằng mạng dự đoán có thể là bất kỳ mô hình phân loại nào, chẳng hạn một mạng nơ-ron sâu hoặc một mô hình phân loại truyền thống hơn. Toàn bộ mô hình sau đó có thể được huấn luyện hoặc tinh chỉnh theo cách của một mô hình phân loại chuẩn. Ví dụ, mạng dự đoán có thể chỉ đơn giản là một tầng Softmax, và các tham số mô hình có thể được tối ưu bằng cách cực đại hóa xác suất của các nhãn đúng.

Chú giải của người dịch: Mạng dự đoán đơn giản nhất

Giả sử bài toán phân loại bình luận thành 3 nhãn {tích cực, trung tính, tiêu cực}. Mạng dự đoán có thể chỉ là: nhân $\mathbf{h}_{\text{cls}}$ (768 số) với một ma trận trọng số để được 3 điểm số, ví dụ $(2.0;\ 0.5;\ -1.0)$, rồi đưa qua Softmax để thành xác suất: $e^{2} \approx 7.39$, $e^{0.5} \approx 1.65$, $e^{-1} \approx 0.37$, tổng ≈ 9.41, nên xác suất là khoảng $(0.79;\ 0.18;\ 0.04)$. Mô hình chọn "tích cực". Lớp gắn thêm này chỉ có $768 \times 3 + 3 = 2\,307$ tham số mới, rất nhỏ so với 110 triệu tham số của BERT.

- **Phân loại** (cặp văn bản). Phân loại cũng có thể được thực hiện trên một cặp văn bản. Giả sử ta có hai văn bản $x_1 \cdots x_m$ và $y_1 \cdots y_n$. Ta có thể nối hai văn bản này để tạo thành một chuỗi duy nhất có độ dài len. Sau đó, ta dự đoán một nhãn cho chuỗi văn bản kết hợp này dựa trên vectơ $\mathbf{h}_{\text{cls}}$, như trong Hình 7.9. Phân loại cặp văn bản bao gồm



Hình 7.9: Phân loại dựa trên BERT cho một cặp văn bản. Ở đây, len $= n + m + 2$. (*Text 1/Text 2*: văn bản 1/văn bản 2; *Class*: lớp.)

một số bài toán, như đánh giá tương đương ngữ nghĩa (*semantic*

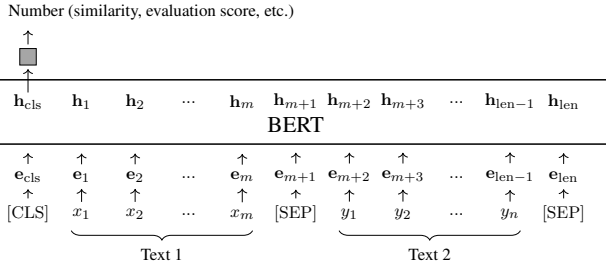
equivalence) (xác định hai văn bản có tương đương về ngữ nghĩa hay không) [Dolan và Brockett, 2005], đánh giá kéo theo văn bản (*text entailment*) (xác định một giả thuyết có thể được suy ra hoặc kéo theo một cách logic từ một tiền đề hay không) [Bentivogli và Giampiccolo, 2011; Williams và cộng sự, 2018], lập luận thường thức có căn cứ (*grounded commonsense inference*) (xác định một sự kiện có khả năng xảy ra hay không khi biết ngữ cảnh của nó) [Zellers và cộng sự, 2018], và lập luận hỏi–đáp (xác định một câu trả lời có tương ứng với một câu hỏi cho trước hay không).

Chú giải của người dịch: Ví dụ kéo theo văn bản

Tiền đề: "Lan đã đỗ đại học Bách khoa." Giả thuyết: "Lan đã thi đại học." $\Rightarrow$ nhãn *kéo theo* (giả thuyết chắc chắn đúng nếu tiền đề đúng). Nếu giả thuyết là "Lan học ngành Công nghệ thông tin" thì nhãn là *trung lập* (không suy ra được), còn "Lan chưa từng thi đại học" thì nhãn là *mâu thuẫn*. Giống mệnh đề kéo theo $P \Rightarrow Q$ ở lớp 10, nhưng ở đây máy phải dựa vào hiểu biết ngôn ngữ và thường thức chứ không phải ký hiệu logic.

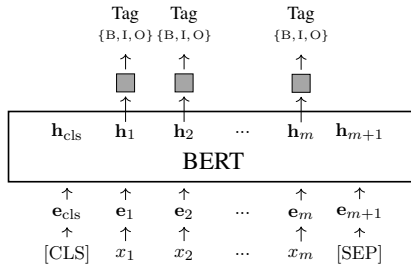
- **Hồi quy** (*Regression*). Thay vì sinh ra một phân phối nhãn, ta có thể cho mạng dự đoán xuất ra một điểm số thực. Ví dụ, bằng cách thêm một tầng Sigmoid vào mạng dự đoán, hệ thống có thể được dùng để tính độ tương đồng giữa hai câu cho trước. Kiến trúc giống như của các hệ thống phân loại dựa trên BERT, chỉ có tầng đầu ra là thay đổi, như trong Hình 7.10. Khi huấn luyện hoặc tinh chỉnh, ta có thể cực tiểu hóa hàm mất mát hồi quy của đầu ra mô hình như thường lệ.

- **Gán nhãn chuỗi** (*Sequence Labeling*). Gán nhãn chuỗi (*sequence labeling*) là một cách tiếp cận học máy áp dụng được cho nhiều bài toán NLP. Cách tiếp cận này gán một nhãn cho mỗi token trong chuỗi đầu vào, rồi từ chuỗi nhãn này có thể suy ra một số chú thích ngôn ngữ học. Một ví dụ về gán nhãn chuỗi trong NLP là gán nhãn từ loại (*part-of-speech (POS) tagging*): ta gán cho mỗi từ trong câu nhãn từ loại tương ứng. Một ví dụ khác là nhận dạng thực thể có tên (*named entity recognition*) (NER), trong đó ta gán cho mỗi từ một nhãn NER, và các thực thể có tên được xác định nhờ những nhãn này. Xem Hình 7.11 để thấy minh họa kiến trúc mô hình cho



Hình 7.10: Hồi quy dựa trên BERT cho một cặp văn bản. (*Number (similarity, evaluation score, etc.)*: một con số (độ tương đồng, điểm đánh giá, v.v.).)

NER. Trong mô hình này, ta dùng lược đồ gán nhãn BIO (*BIO*



Hình 7.11: Gán nhãn chuỗi dựa trên BERT cho nhận dạng thực thể có tên. Ở đây $\{\text{B}, \text{I}, \text{O}\}$ là tập nhãn của NER. (*Tag*: nhãn.)

tagging scheme) để biểu diễn các thực thể có tên. Ví dụ, B-ORG nghĩa là phần bắt đầu của một tổ chức, I-ORG nghĩa là từ nằm bên trong một tổ chức, và O nghĩa là từ không thuộc thực thể có tên nào. Mô hình NER này có thể xuất ra một phân phối trên tập nhãn tại mỗi vị trí, ký hiệu là $\mathbf{p}_i$. Việc huấn luyện hoặc tinh chỉnh mô hình có thể được thực hiện trên các phân phối $\{\mathbf{p}_1, \cdots, \mathbf{p}_m\}$ này. Ví dụ, giả sử $p_i(\text{tag}_i)$ là xác suất của nhãn đúng tại vị trí i. Hàm mất mát huấn luyện có thể được định nghĩa là log-hợp lý âm (*negative log-likelihood*)

$$\text{Loss} = -\frac{1}{m}\sum_{i=1}^{m} \log p_i(\text{tag}_i). \tag{7.23}$$

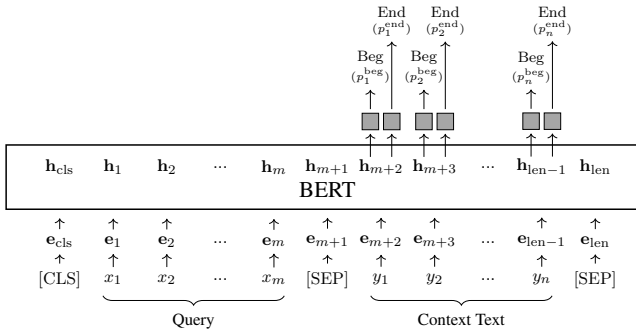
Việc tìm chuỗi nhãn tốt nhất khi đã có một mô hình NER được huấn luyện là một bài toán được nghiên cứu kỹ trong NLP. Điều này thường được thực hiện bằng quy hoạch động (*dynamic programming*), phương pháp mà, trong bối cảnh tìm đường trên một lưới (*lattice*), có độ phức tạp tuyến tính [Huang, 2009].

Chú giải của người dịch: Ví dụ gán nhãn BIO

Câu "Nguyễn Văn An làm việc tại Đại học Bách khoa Hà Nội" (tách theo âm tiết) được gán nhãn: Nguyễn/B-PER Văn/I-PER An/I-PER làm/O việc/O tại/O Đại/B-ORG học/I-ORG Bách/I-ORG khoa/I-ORG Hà/I-ORG Nội/I-ORG (PER = người, ORG = tổ chức). Nhãn B đánh dấu chữ *bắt đầu* một thực thể, I là chữ *bên trong* thực thể đó, O là *ngoài* mọi thực thể. Đọc chuỗi nhãn, ta lấy lại được hai thực thể: một tên người và một tên tổ chức. Phương trình (7.23) đơn giản là trung bình "điểm phạt" $-\log p$ trên m vị trí; chia cho m để câu dài hay ngắn đều có thang đo như nhau.

- **Dự đoán đoạn** (*Span Prediction*). Một số nhiệm vụ NLP đòi hỏi dự đoán một đoạn (*span*) trong văn bản. Một ví dụ phổ biến là đọc hiểu (*reading comprehension*). Trong nhiệm vụ này, ta được cho một truy vấn $x_1 \cdots x_m$ và một văn bản ngữ cảnh $y_1 \cdots y_n$. Mục tiêu là xác định một đoạn liên tục trong $y_1 \cdots y_n$ trả lời tốt nhất truy vấn đó. Bài toán này có thể được đặt dưới dạng một nhiệm vụ tương tự gán nhãn chuỗi, trong đó ta dự đoán một nhãn cho mỗi y_j để chỉ ra điểm bắt đầu hoặc kết thúc của đoạn. Theo Seo và cộng sự [2017], ta thêm hai mạng bên trên đầu ra BERT cho y_j: một mạng sinh xác suất để y_j là điểm bắt đầu của đoạn (ký hiệu là p_j^{beg}), và một mạng sinh xác suất để y_j là điểm kết thúc của đoạn (ký hiệu là p_j^{end}). Kiến trúc mô hình thu được được trình bày trong Hình 7.12.
  Ta ghép truy vấn và văn bản ngữ cảnh lại để được chuỗi đầu vào. Các mạng dự đoán chỉ được áp dụng cho các đầu ra ứng với văn bản ngữ cảnh, sinh ra các xác suất p_j^{beg} và p_j^{end} tại mỗi vị trí. Hàm mất mát có thể được tính bằng cách cộng log-hợp lý âm của hai mô hình trên toàn bộ văn bản ngữ cảnh.

$$\text{Loss} \;=\; -\frac{1}{n}\sum_{j=1}^{n}\left(\log p_j^{\text{beg}} + \log p_j^{\text{end}}\right). \tag{7.24}$$



Hình 7.12: Dự đoán đoạn dựa trên BERT cho đọc hiểu. (*Query*: truy vấn; *Context Text*: văn bản ngữ cảnh; *Beg/End*: bắt đầu/kết thúc.)

Khi kiểm tra, ta tìm đoạn tốt nhất bằng

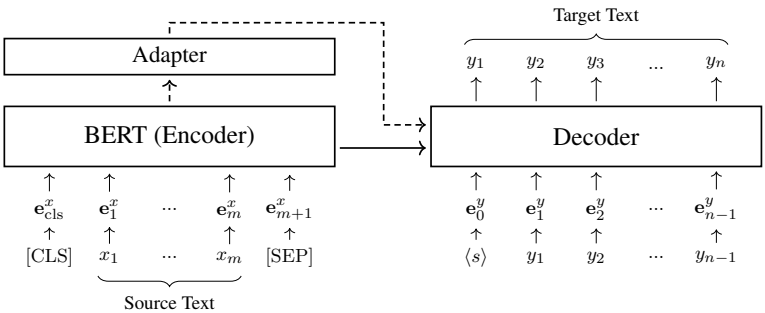
$$(\hat{j}_1, \hat{j}_2) = \underset{1 \le j_1 \le j_2 \le n}{\arg\max} \left(\log p_{j_1}^{\text{beg}} + \log p_{j_2}^{\text{end}} \right). \tag{7.25}$$

Chú giải của người dịch: Tìm đoạn trả lời

Ví dụ: truy vấn "Thủ đô của Việt Nam là gì?", văn bản ngữ cảnh "Hà Nội là thủ đô của Việt Nam .". Mô hình cho mỗi chữ trong ngữ cảnh hai xác suất: làm điểm *bắt đầu* và làm điểm *kết thúc* của câu trả lời. Nếu p^{beg} lớn nhất ở "Hà" và p^{end} lớn nhất ở "Nội" thì câu trả lời là đoạn "Hà Nội". Phương trình (7.25) thử mọi cặp (bắt đầu j_1, kết thúc j_2) với điều kiện $j_1 \le j_2$ (điểm bắt đầu không thể nằm sau điểm kết thúc) và chọn cặp có tổng log-xác suất lớn nhất; vì log của tích bằng tổng các log, đó cũng là cặp có tích $p_{j_1}^{\text{beg}} \cdot p_{j_2}^{\text{end}}$ lớn nhất.

- **Mã hóa cho các mô hình mã hóa–giải mã** (*Encoding for Encoder-Decoder Models*). Mặc dù trọng tâm của mục này chủ yếu là các bài toán hiểu ngôn ngữ, cần lưu ý rằng mô hình BERT có thể được áp dụng cho nhiều loại nhiệm vụ NLP hơn. Thực tế, mô hình BERT có thể được dùng trong mọi tình huống mà ta cần mã hóa một đoạn văn bản. Một ứng dụng mà ta chưa đề cập là sinh văn bản (*text generation*), bao gồm nhiều nhiệm vụ như dịch máy (*machine translation*), tóm tắt (*summarization*), hỏi đáp (*question answering*) và sinh hội thoại (*dialogue generation*). Những nhiệm vụ này có thể

được phát biểu dưới dạng các bài toán chuỗi-sang-chuỗi (*sequence-to-sequence*): ta dùng một bộ mã hóa để biểu diễn văn bản nguồn và một bộ giải mã (*decoder*) để sinh văn bản đích tương ứng. Một cách đơn giản để áp dụng mô hình BERT là xem chúng như các bộ mã hóa. Trước khi tinh chỉnh, ta có thể khởi tạo tham số của bộ mã hóa bằng tham số của một mô hình BERT đã tiền huấn luyện. Sau đó, mô hình mã hóa–giải mã có thể được tinh chỉnh trên các cặp văn bản như thường lệ. Hình 7.13 trình bày kiến trúc của một hệ thống dịch máy nơ-ron (*neural machine translation*) trong đó một mô hình BERT được áp dụng ở phía nguồn.



Hình 7.13: Dùng BERT làm bộ mã hóa trong một mô hình mã hóa–giải mã. Ở đây $x_1 \cdots x_m$ ký hiệu chuỗi nguồn, $y_1 \cdots y_n$ ký hiệu chuỗi đích, $\mathbf{e}^x_1 \cdots \mathbf{e}^x_m$ ký hiệu chuỗi vectơ nhúng của $x_1 \cdots x_m$, và $\mathbf{e}^y_1 \cdots \mathbf{e}^y_n$ ký hiệu chuỗi vectơ nhúng của $y_1 \cdots y_n$. Bộ tiếp hợp (*adapter*), vốn là tùy chọn, ánh xạ đầu ra của mô hình BERT sang dạng phù hợp hơn với bộ giải mã. (*Source Text*: văn bản nguồn; *Target Text*: văn bản đích; $\langle s \rangle$: token bắt đầu chuỗi.)

Tinh chỉnh mô hình BERT là một bài toán kỹ thuật phức tạp, chịu ảnh hưởng của nhiều yếu tố, như lượng dữ liệu tinh chỉnh, kích thước mô hình và bộ tối ưu (*optimizer*) dùng khi tinh chỉnh. Nói chung, ta mong muốn tinh chỉnh các mô hình này đủ để chúng hoạt động tốt trên các nhiệm vụ hạ nguồn. Tuy nhiên, việc tinh chỉnh mô hình BERT cho các nhiệm vụ cụ thể có thể dẫn đến quá khớp (*overfitting*), từ đó làm giảm khả năng tổng quát hóa (*generalize*) sang các nhiệm vụ khác. Ví dụ, giả sử ta có một mô hình BERT hoạt động tốt trên một nhiệm vụ cụ thể. Nếu sau đó ta tinh chỉnh nó cho các nhiệm vụ mới, hiệu năng của nó trên nhiệm vụ ban đầu có thể giảm xuống. Vấn đề này liên quan tới bài toán **quên thảm khốc**

(*catastrophic forgetting*) trong huấn luyện liên tục (*continual training*), trong đó một mạng nơ-ron quên đi thông tin đã học trước đó khi được cập nhật trên các mẫu mới. Trong các ứng dụng thực tế, một cách phổ biến để giảm nhẹ hiện tượng quên thảm khốc là thêm một số dữ liệu cũ vào quá trình tinh chỉnh và huấn luyện mô hình với dữ liệu đa dạng hơn. Ngoài ra, ta có thể dùng các phương pháp chuyên biệt cho quên thảm khốc, như phát lại kinh nghiệm (*experience replay*) [Rolnick và cộng sự, 2019] và củng cố trọng số đàn hồi (*elastic weight consolidation*) [Kirkpatrick và cộng sự, 2017]. Bạn đọc quan tâm có thể tham khảo một số bài tổng quan để có thảo luận chi tiết hơn về vấn đề này trong học liên tục (*continual learning*) [Parisi và cộng sự, 2019; Wang và cộng sự, 2023a;f].

Chú giải của người dịch: Quên thảm khốc

Hãy tưởng tượng một học sinh ôn thi khối A rất kỹ, rồi trong ba tháng chỉ học tiếng Pháp; khi quay lại làm đề Toán, bạn ấy có thể quên nhiều. Mạng nơ-ron còn "quên" nặng hơn: khi các tham số bị điều chỉnh để phục vụ nhiệm vụ mới, chính những tham số đó vốn đang lưu kiến thức cho nhiệm vụ cũ. Cách chữa "trộn lại dữ liệu cũ" giống việc vừa học tiếng Pháp vừa thỉnh thoảng làm lại vài đề Toán.

7.5 Tóm tắt

Trong chương này, chúng ta đã giới thiệu khái niệm chung về tiền huấn luyện trong NLP. Cụ thể, chúng ta đã thảo luận về tiền huấn luyện tự giám sát và ứng dụng của nó cho các kiến trúc chỉ có bộ mã hóa, chỉ có bộ giải mã, và mã hóa–giải mã. Hơn nữa, chúng ta đã trình bày và so sánh nhiều nhiệm vụ tiền huấn luyện cho các kiến trúc này. Như một ví dụ, chúng ta đã dùng BERT để minh họa cách các mô hình chuỗi được tiền huấn luyện thông qua mô hình hóa ngôn ngữ có che và được áp dụng cho các nhiệm vụ hạ nguồn khác nhau.

Những năm gần đây đã chứng kiến những tiến bộ đáng kể trong NLP, dẫn dắt bởi việc sử dụng tiền huấn luyện tự giám sát ở quy mô lớn. Những bước tiến quan trọng đã đạt được trên nhiều nhiệm vụ, không chỉ trong NLP mà cả trong thị giác máy tính (*computer vision*) và các lĩnh vực khác của trí tuệ nhân tạo (*artificial intelligence*) (AI). Một ý tưởng đằng sau những tiến bộ này là một lượng lớn tri thức về thế giới có thể được học

chỉ bằng cách huấn luyện các hệ thống AI này trên những lượng khổng lồ dữ liệu không gán nhãn (*unlabeled data*). Ví dụ, một mô hình ngôn ngữ có thể học được một số tri thức chung về một ngôn ngữ bằng cách lặp đi lặp lại việc dự đoán các từ bị che trong văn bản quy mô lớn. Kết quả là, mô hình ngôn ngữ đã tiền huấn luyện này có thể đóng vai trò một mô hình nền tảng (*foundation model*), có thể dễ dàng được thích ứng để giải các nhiệm vụ NLP hạ nguồn cụ thể. Sự chuyển dịch mô thức (*paradigm shift*) này trong NLP đã cho phép phát triển những hệ thống mạnh mẽ cho hiểu ngôn ngữ, sinh ngôn ngữ và lập luận (*reasoning*) [Manning, 2022]. Tuy nhiên, điều quan trọng là phải nhận thức rằng chúng ta vẫn đang ở giai đoạn đầu của việc tạo ra những hệ thống thực sự thông minh, và còn một chặng đường dài phía trước. Dù vậy, tiền huấn luyện quy mô lớn đã mở ra cánh cửa tới những hệ thống thông minh mà các nhà nghiên cứu từ lâu đã mong muốn phát triển, mặc dù vẫn còn một số hướng nghiên cứu then chốt đang bỏ ngỏ để khám phá, chẳng hạn học hiệu quả từ dữ liệu hạn chế, và có được năng lực lập luận và lập kế hoạch (*planning*) phức tạp.

Lưu ý rằng chương này chủ yếu mang tính nhập môn và không thể bao quát mọi khía cạnh của tiền huấn luyện. Ví dụ, có nhiều phương pháp để tinh chỉnh một mô hình tiền huấn luyện, mang lại những cách khác nhau để thích ứng mô hình với các tình huống đa dạng. Hơn nữa, các mô hình ngôn ngữ lớn, vốn được xem là một trong những thành tựu quan trọng nhất của AI trong những năm gần đây, chưa được đề cập trong chương này. Chúng tôi để dành việc thảo luận các chủ đề này cho các chương tiếp theo.

Bản dịch tiếng Việt có chú giải (không chính thức, phi thương mại) của *Natural Language Processing: Neural Networks and Large Language Models*, © 2021–2026 Tong Xiao và Jingbo Zhu, `https://github.com/NiuTrans/NLPBook`. Nguyên tác và bản dịch theo giấy phép CC BY-NC 4.0.